r3ditor

Open-Source CAD/CAM Editor

Technical Documentation

Architecture & Engineering Reference — v0.2.0

15	295	23	37
Rust Crates	Rust Tests	Materials	Tauri Commands

Rust 1.93 · Truck 0.6 · Tauri 2.2 · React 18.3 · Three.js 0.170 · Zustand 5.0

License: AGPL-3.0 \ Generated: June 2025 \ Classification: Public

Table of Contents

Executive Summary ... 6
- Vision ... 6
- Key Capabilities ... 6
  - Pure-Rust CAD Kernel ... 6
  - Sketch System (Blender-Inspired) ... 6
  - Integrated Manufacturing ... 6
  - Cross-Platform Deployment ... 6
- Architecture at a Glance ... 7
- Project Metrics ... 7

Architecture Overview ... 9
- System Topology ... 9
- Crate Organization ... 9
  - Layer 1 — Foundation ... 9
  - Layer 2 — Core Engines ... 10
  - Layer 3 — Integration ... 10
  - Layer 4 — Applications ... 11
- ECS Frame Schedule ... 11
- Data Flow ... 12

CAD Kernel ... 13
- Dual Kernel Strategy ... 13
- Primary Kernel — Truck B-Rep ... 13
- Parametric Feature Tree ... 14
  - Supported Feature Types ... 14
  - Feature Tree Data Model ... 14
  - History & Undo/Redo ... 15
- Constraint Solver ... 15
  - 2D Sketch Constraints ... 15
  - 4-Stage Cascade Solver ... 15
  - 3D Assembly Constraints ... 16
- B-Rep Operations ... 16
  - Boolean Operations ... 16
  - Tessellation ... 16
- OpenCASCADE Fallback ... 16
- Sketch Operations System ... 17
  - Sketch Entities ... 17
  - Sketch Tools (StatefulTool Pattern) ... 17
  - Snap Engine ... 18
  - Advanced Operations (sketch_ops.rs) ... 18
  - Snapshot System ... 18

CAM Engine ... 19
- CNC Physics Models ... 19
  - Cutting Force — Kienzle Model ... 19
  - Tool Life — Taylor Equation ... 19
  - Cutting Temperature — Loewen-Shaw ... 20

© 2026 — MIT OR Apache-2.0

- Chatter Stability — Altintas Model ... 20
- Material Removal Rate ... 20
- Surface Finish Prediction ... 20
- Sheet Metal Operations ... 20
  - Cutting Methods ... 20
  - Bending Models ... 20
- Material Catalogs ... 21
  - Sheet Materials (17) ... 21
  - CNC Materials (6) ... 21
- Toolpath Generation ... 22
  - G-code Post-Processors ... 22
- Nesting Optimization ... 22

GPU Acceleration & Renderer ... 23
- Current Implementation Status ... 23
- Target Rendering Architecture ... 23
- 11-Pass Render Pipeline ... 24
- PBR Shading Model ... 24
- WGSL Shader Modules ... 24
  - Shader Budget ... 25
- Visual Modes ... 25
- GPU Compute Strategy ... 25
- Camera System ... 26
  - 3-Point Lighting ... 26
- Uniform Buffer Layout ... 26

Editor Shell & UI ... 27
- Dual UI Strategy ... 27
  - Three.js — 3D Viewport ... 27
  - React + Tailwind — Panels ... 27
- Editor Shell Architecture ... 27
  - Core Components ... 27
  - EditorApp Lifecycle ... 28
- Command System ... 29
- Keyboard Shortcuts ... 30
- React Frontend ... 30
  - Component Architecture ... 30
  - State Management ... 31
  - Tauri IPC Bridge ... 31
  - Technology Stack ... 31

API Gateway & Cloud Services ... 32
- API Gateway ... 32
  - Route Map ... 32
  - Authentication ... 32
  - Application State ... 33
- Worker Services ... 33
  - Worker Architecture ... 33
  - Redis Stream Protocol ... 33
- Database Schema ... 34
- Object Storage ... 34
- API Documentation ... 34
- Error Handling ... 35

Plugin System ... 36
- Architecture ... 36

© 2026 — MIT OR Apache-2.0

Plugin Categories ... 36
Plugin Manifest ... 37
Security Model ... 37
Capability-Based Access ... 37
Sandbox Guarantees ... 38
Plugin Host API ... 38
Registry & Discovery ... 39
Example: Custom Material Plugin ... 39
Data Formats & File I/O ... 40
Native Format: .manu ... 40
Binary Mesh Format ... 40
Import Formats ... 41
Export Formats ... 41
G-Code Post-Processors ... 42
File I/O Pipeline ... 42
Import Pipeline ... 42
Export Pipeline ... 43
Deployment & Distribution ... 44
Deployment Modes ... 44
Desktop Distribution ... 45
Tauri 2.2 Packaging ... 45
Auto-Update ... 45
Cloud Deployment ... 45
Docker Compose (Development / Small Teams) ... 45
Docker Compose File ... 46
Kubernetes (Production) ... 47
Horizontal Scaling Strategy ... 47
Platform Adapters ... 47
Performance Engineering ... 49
Performance Targets ... 49
Geometry Operations ... 49
Rendering ... 50
Manufacturing ... 50
Memory Budget ... 50
Parallelism Architecture ... 51
Thread Budget ... 51
Rayon Work-Stealing ... 51
Tokio Async I/O ... 51
Benchmarking ... 52
Profiling Toolkit ... 52
Migration Path ... 54
Overview ... 54
Phase 1: Foundation Sprint (Weeks 1–4) ... 54
Phase 2: Geometry Engine (Weeks 5–10) ... 55
Phase 3: Manufacturing Intelligence (Weeks 11–16) ... 55
Phase 4: Desktop & Plugins (Weeks 17–22) ... 56
Phase 5: Cloud & Marketplace (Weeks 23–30) ... 57
Risk Mitigation ... 57
Appendices ... 59
Appendix A: Workspace Crate Reference ... 59
Appendix B: Technology Matrix ... 60

© 2026 — MIT OR Apache-2.0

Appendix C: Database Schema 61
Appendix D: Docker Services Inventory 63
Appendix E: Platform Adapter Configuration 63
Shopify Adapter 63
WooCommerce Adapter 64
Appendix F: Keyboard Shortcuts Reference 64
Appendix G: Glossary 65

© 2026 — MIT OR Apache-2.0

Executive Summary

Vision

r3ditor is a **full-featured open-source 3D CAD/CAM editor** built from the ground up in Rust, targeting professional-grade manufacturing workflows. It transforms an existing DFM Quote Suite — a browser-based upload ⟶ analyze ⟶ quote pipeline — into an integrated design-to-manufacture platform.

i Mission Statement

Deliver an enterprise-grade parametric 3D modeling tool with integrated CNC/sheet-metal manufacturing capabilities, GPU-accelerated rendering, and cross-platform deployment — all powered by a pure-Rust crate ecosystem.

Key Capabilities

Pure-Rust CAD Kernel

Built on the **Truck** B-Rep ecosystem with NURBS surface modeling, 20 parametric feature kinds, boolean operations, topological naming (5-priority cascade), and full transaction-based undo/redo history.

Sketch System (Blender-Inspired)

7 entity types, 19 constraint types, 4-stage cascade solver (DogLeg⟶LM⟶BFGS⟶NR), stateful tool framework with continuous draw, snap engine (7 types), GPU picking color map, sketch snapshots, and clipboard with dependency resolution.

Integrated Manufacturing

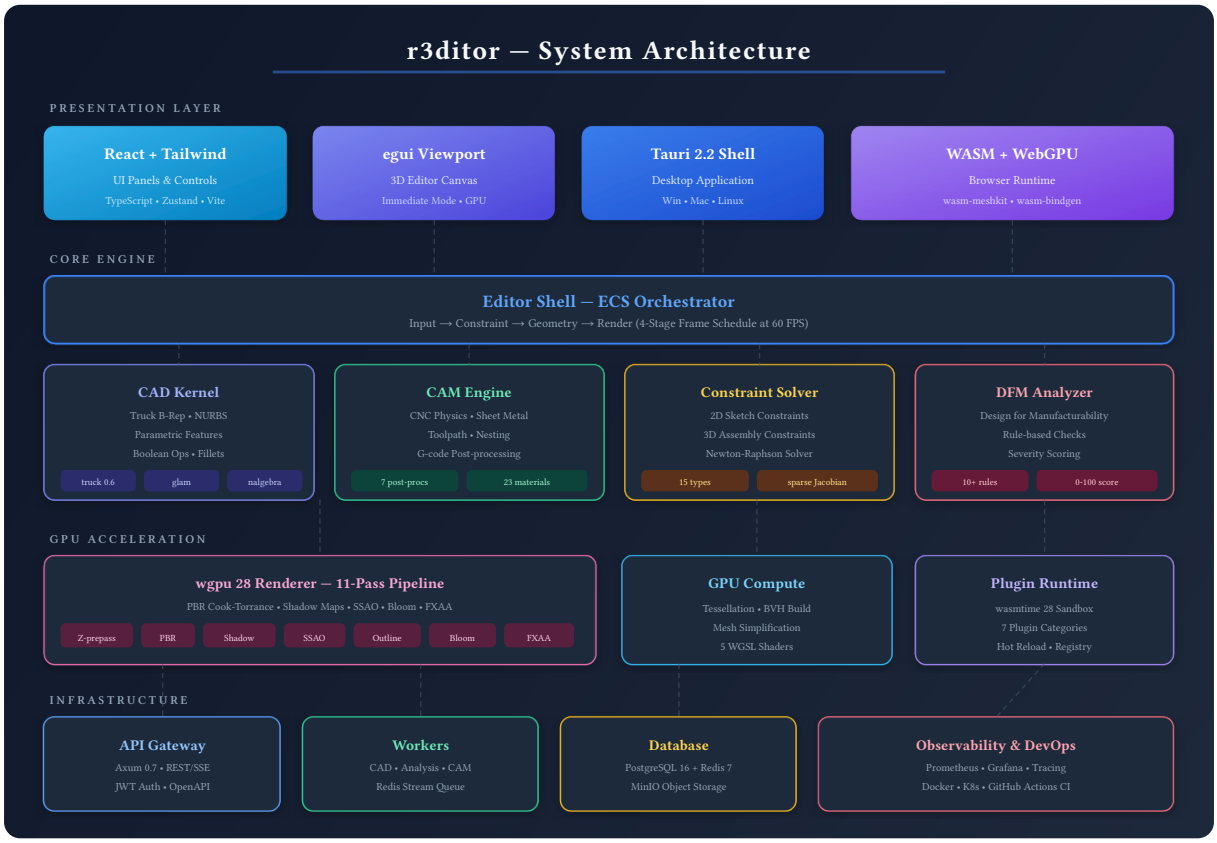
CNC toolpath generation with physics models (Kienzle, Taylor, Loewen-Shaw, Altintas), sheet metal cutting (laser/plasma/waterjet), bending with springback prediction, 8 G-code post-processors, and 2D nesting optimization.

Cross-Platform Deployment

Native desktop via **Tauri 2.2** with React 18.3/TypeScript 5.6/Three.js 0.170 frontend, 37 Tauri IPC commands, Zustand 5.0 state management. Cloud mode via Axum API + Docker/Kubernetes.

© 2026 — MIT OR Apache-2.0

Architecture at a Glance



Project Metrics

15	295	319	23	8
Rust Crates	Rust Tests	Frontend Tests	Materials	Post-Processors

19	20	37	22	21
Constraint Types	Feature Types	Tauri Commands	Editor Commands	Tool Types

== Design Principles

The architecture is guided by eight core principles:

Principle	Description
Zero-Copy GPU Path	Geometry flows from B-Rep evaluation → tessellation → GPU buffer with minimal intermediate allocation.

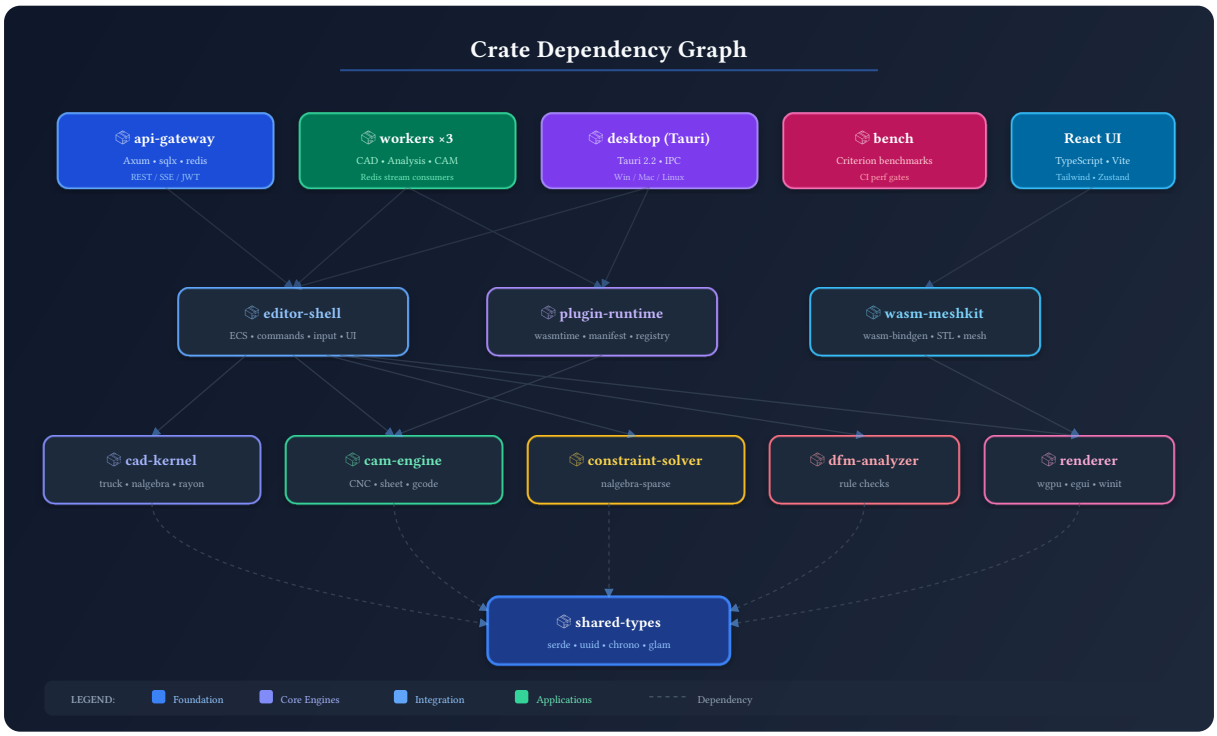
Principle	Description
Rayon Everywhere	Data-parallel operations (tessellation, nesting, DFM checks) use Rayon's work-stealing thread pool.
ECS-Inspired Schedule	4-stage frame loop (Input → Constraint → Geometry → Render) ensures deterministic execution order.
Type-Safe Boundaries	`shared-types` crate defines all API contracts, preventing deserialization mismatches across 15 crates.
Plugin Sandbox	WASM plugins run in wasmtime with capability-based security — untrusted code cannot access the filesystem.
Offline-First	Desktop mode works fully offline; cloud features are additive enhancements, not requirements.
Incremental Migration	Each phase adds capability while preserving the existing working system.
Manufacturing Truth	Physics-based models (Kienzle, Taylor) ensure cost estimates reflect real-world machining conditions.

© 2026 — MIT OR Apache-2.0

Architecture Overview

System Topology

The r3ditor system is organized as a **Rust workspace** containing 15 crates, a React frontend, and supporting infrastructure. The architecture follows a layered design with clear dependency boundaries enforced by the Cargo workspace.



Crate Organization

The workspace is organized into four dependency layers:

Layer 1 — Foundation

shared-types serves as the foundation crate, defining all cross-cutting data types, API contracts, and domain enumerations. Every other crate depends on it.

Module	Contents	Key Types
api.rs	REST API request/response types	UploadResponse, JobStatus, QuoteResult
geometry.rs	3D geometry primitives	Point3, Vector3, BoundingBox, Mesh
materials.rs	Material catalogs (17 sheet + 6 CNC)	SheetMaterial, CncMaterial, PhysicalProperties
dfm.rs	Design-for-manufacturing types	DfmFinding, Severity, DfmScore
manufacturing.rs	Manufacturing process definitions	CuttingMethod, BendingMethod, CncOperation
estimation.rs	Cost estimation models	SheetEstimate, CncEstimate, QuoteLineItem
units.rs	Physical unit conversions	Length, Force, Temperature, Pressure
platform.rs	Platform adapter interfaces	PlatformAdapter, ShopifyConfig

Layer 2 — Core Engines

These crates implement the domain-specific computation:

cad-kernel

Truck-based B-Rep kernel with NURBS surfaces, boolean operations, parametric feature tree (20 feature types), and history-based undo/redo. Includes a full **sketch system** with 7 entity types, trim/bevel/offset operations, StatefulTool machine, snap engine, and picking color map. 13 modules, 10,269 lines, 125 tests.

constraint-solver

2D sketch constraint solver with **19 constraint types** and 3D assembly constraint solver. Uses a **4-stage cascade solver** (DogLeg ⟶ Levenberg-Marquardt ⟶ BFGS ⟶ Newton-Raphson) with sparse Jacobian matrices via nalgebra-sparse. 4 files, 1,915 lines, 30 tests.

renderer

Three.js-based rendering infrastructure providing scene management, camera controller (orbit/pan/zoom), and mesh display. Currently 553 lines of foundational code — advanced GPU pipeline planned for future phases.

cam-engine

CNC physics (6 models), sheet metal cutting (4 methods), bending (3 formulas), toolpath generation, nesting optimization, and G-code post-processing for **8 machine types** (Fanuc, Haas, Mazak, LinuxCNC, Grbl, Marlin, Klipper, Heidenhain). 6 files, 1,555 lines, 64 tests.

dfm-analyzer

Design-for-manufacturability analysis with rule-based checks, severity scoring (0–100), and actionable feedback. Pluggable rule system for custom industry checks. 3 files, 561 lines, 27 tests.

Layer 3 — Integration

editor-shell

Orchestrator that wires all engines

plugin-runtime

wasmtime 28 plugin host with

wasm-meshkit

WASM-compiled mesh toolkit ex-

together via the World struct. Manages 22 EditorCommand variants, 21 tool types (7 sketch tools with StatefulTool lifecycle), sketch mode, ToolSnapshotManager, ClipboardBuffer, and the Tauri IPC bridge. 6 files, 1,301 lines, 22 tests.

manifest parsing, capability-based security, registry management, and hot-reload support for 7 plugin categories.

posing STL parsing, mesh simplification (edge collapse), and geometry queries to browser environments via wasm-bindgen.

Layer 4 — Applications

api-gateway — Axum 0.7 REST API with JWT auth, file upload, job management via Redis streams, material queries, and quote generation. Includes SSE for real-time job status.

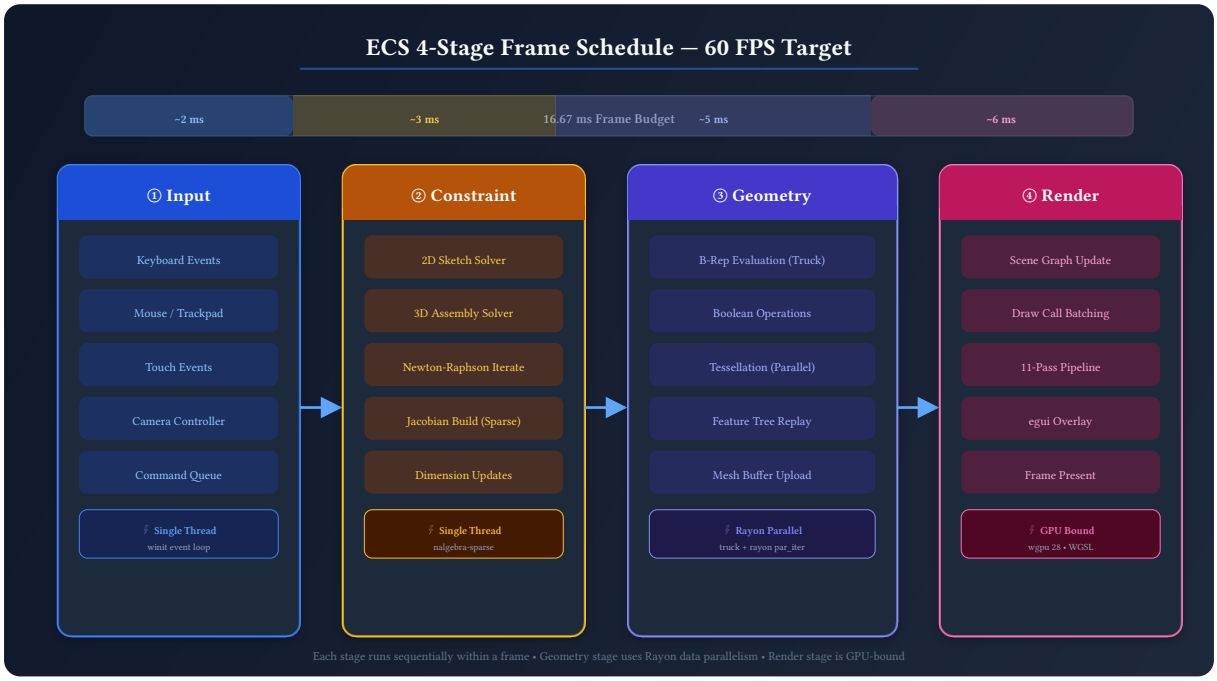
Desktop App (apps/desktop) — Tauri 2.2 shell with **37 IPC commands** (commands.rs, 692 lines), React 18.3 + TypeScript 5.6 frontend (1,062-line Zustand store), and native file system access.

worker-analysis / worker-cad / worker-cam — Redis stream consumers for DFM analysis, geometry import, and toolpath/nesting computation respectively.

bench — Criterion benchmark suite covering tessellation, constraint solving, CNC estimation, nesting, and DFM analysis with CI performance gates.

ECS Frame Schedule

The editor operates on a **4-stage frame schedule**, targeting a 16.67 ms budget (60 FPS):



Stage	Budget	Responsibilities	Threading
① Input	2 ms	Poll keyboard, mouse, touch events; update camera; enqueue commands	Single thread

Stage	Budget	Responsibilities	Threading
② Constraint	3 ms	Solve 2D sketch constraints via 4-stage cascade (DogLeg → LM → BFGS → NR); update dimension values	Single thread
③ Geometry	5 ms	Evaluate B-Rep features; boolean ops; tessellation; upload mesh buffers	Rayon parallel
④ Render	6 ms	Update scene graph; batch draw calls; run 11-pass pipeline; present frame	GPU-bound

i Key Design Decision

Stages run **sequentially within a frame** to ensure deterministic behavior. Only the Geometry stage uses Rayon data parallelism internally, as B-Rep tessellation is embarrassingly parallel across faces.

Data Flow

The data flows through the system in a well-defined pipeline:

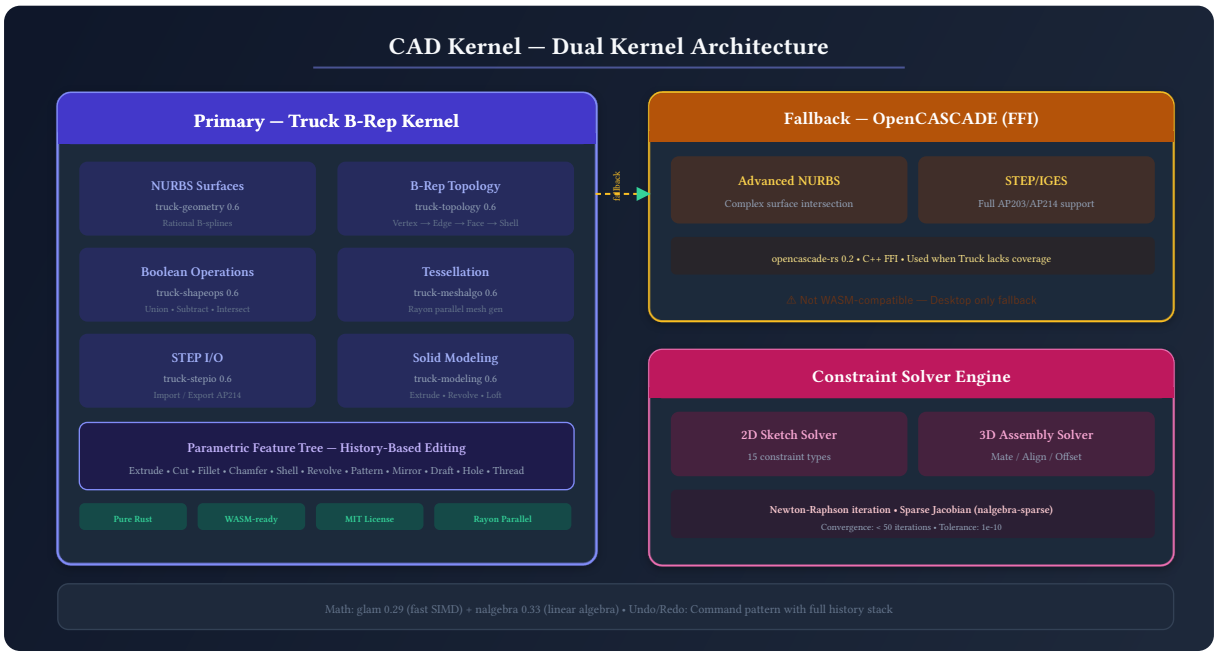
1. **User Input** → captured by React frontend → dispatched via Tauri IPC (`invoke`) → `EditorCommand` queue
2. **Commands** → processed by editor shell (`World` struct) → modify `FeatureTree`, `ConstraintSet`, and `SketchState`
3. **Constraint Solver** → resolves sketch constraints via 4-stage cascade → updates parametric dimensions
4. **CAD Kernel** → replays feature tree → evaluates B-Rep geometry → produces `BRepModel`
5. **Tessellation** → converts NURBS surfaces → triangle mesh (parallel via Rayon)
6. **Frontend Sync** → mesh data → Tauri events → Three.js scene update
7. **Render** → Three.js renders 3D viewport in React webview
8. **DFM Analysis** → runs on demand → returns severity findings
9. **CAM Engine** → generates toolpaths → produces G-code via 8 post-processors

© 2026 — MIT OR Apache-2.0

CAD Kernel

Dual Kernel Strategy

The r3ditor CAD kernel employs a **dual kernel strategy**: the Truck B-Rep ecosystem serves as the primary kernel (pure Rust, WASM-compatible), while OpenCASCADE provides a fallback for advanced NURBS operations and complex STEP file import/export.



Primary Kernel — Truck B-Rep

The Truck ecosystem provides a complete set of pure-Rust crates for boundary representation modeling:

Crate	Version	Purpose
truck-base	0.6	Core traits and utility types
truck-geotrait	0.6	Geometric trait definitions
truck-geometry	0.6	NURBS curves and surfaces, rational B-splines
truck-topology	0.6	B-Rep topology: Vertex ⟶ Edge ⟶ Wire ⟶ Face ⟶ Shell ⟶ Solid
truck-modeling	0.6	High-level modeling operations (extrude, revolve, loft, sweep)

Crate	Version	Purpose
truck-shapeops	0.6	Boolean operations (union, subtract, intersect)
truck-meshalgo	0.6	Tessellation algorithms, mesh generation from NURBS
truck-polymesh	0.6	Polygon mesh representation and manipulation
truck-stepio	0.6	STEP AP214 file import and export
truck-platform	0.6	GPU rendering integration
truck-rendimpl	0.6	Render implementation for Truck geometry

✓ Why Truck?

Truck is the only production-quality **pure-Rust** B-Rep kernel, making it compilable to WASM for browser deployment. It is MIT-licensed, actively maintained, and provides excellent Rayon integration for parallel tessellation.

Parametric Feature Tree

The CAD kernel implements a **history-based parametric modeling** approach with a feature tree that records all design operations:

Supported Feature Types

Additive
- Extrude (Boss)
- Revolve
- Loft
- Sweep

Subtractive
- Cut (Extrude)
- Hole (Simple / Counterbore)
- Thread
- Shell

Modifiers
- Fillet
- Chamfer
- Draft
- Pattern (Linear / Circular)
- Mirror

Feature Tree Data Model

```
pub enum Feature {
    Extrude { sketch_id: Uuid, depth: f64, direction: Direction },
    Cut { sketch_id: Uuid, depth: f64 },
    Fillet { edge_ids: Vec<Uuid>, radius: f64 },
    Chamfer { edge_ids: Vec<Uuid>, distance: f64 },
    Shell { face_ids: Vec<Uuid>, thickness: f64 },
    Revolve { sketch_id: Uuid, axis: Axis, angle: f64 },
    Pattern { feature_id: Uuid, pattern_type: PatternType },
    Mirror { feature_id: Uuid, plane: Plane },
    Hole { center: Point3, diameter: f64, depth: f64 },
    Thread { hole_id: Uuid, pitch: f64, thread_type: ThreadType },
```

```
    Draft { face_ids: Vec<Uuid>, angle: f64, direction: Vector3 },
}
```

History & Undo/Redo

The kernel implements a **command pattern** for full undo/redo support:

- Every feature operation creates an undoable `EditorCommand`
- The undo stack stores up to **48 MB** of history (configurable)
- Redo is supported until a new operation branches the history
- Feature tree replay reconstructs the B-Rep from any point in history

> ⚠ **Performance Note**
>
> Full feature tree replay evaluates all features from the root. For models with 50+ features, incremental evaluation is used — only features downstream of the edit are re-evaluated.

Constraint Solver

2D Sketch Constraints

The sketch constraint solver supports **19 constraint types**, solved via a 4-stage cascade solver:

Geometric	Dimensional	Relational	Advanced
Coincident	Horizontal Distance	Equal Length	Concentric
Horizontal	Vertical Distance	Symmetric	EqualRadius
Vertical	Angle	Tangent	PointOnLine
Perpendicular	Radius	Midpoint	PointOnCircle
Parallel	Diameter		

4-Stage Cascade Solver

The solver uses a cascading strategy for maximum robustness — each stage acts as a fallback:

Stage	Algorithm	Characteristics
1	DogLeg Trust Region	Default solver; excellent convergence for well-conditioned systems
2	Levenberg-Marquardt	Damped least-squares; handles near-singular Jacobians
3	BFGS Quasi-Newton	Gradient-based; good for large constraint sets
4	Newton-Raphson	Direct solve with line search; final fallback

© 2026 — MIT OR Apache-2.0

Solver parameters:

- **Maximum iterations per stage**: 50
- **Convergence tolerance**: 1×10^{-10}
- **Jacobian**: Sparse matrix via `nalgebra-sparse`
- **Cascade trigger**: stage fails ⟶ next stage inherits current state
- **Total tests**: 30 (constraint-solver crate)

3D Assembly Constraints

For multi-body assemblies, the 3D constraint solver supports:

- **Mate** — coincident faces
- **Align** — parallel axes/planes
- **Offset** — fixed distance between entities
- **Angular** — fixed angle between planes
- **Gear ratio** — linked rotational constraints

B-Rep Operations

Boolean Operations

Boolean operations (union, subtract, intersect) are implemented via `truck-shapeops`:

Operation	Target	Description
Union	< 50 ms (10K faces)	Merge two solids into a single body
Subtract	< 50 ms (10K faces)	Remove one solid from another (cuts, holes)
Intersect	< 50 ms (10K faces)	Retain only the overlapping volume

Tessellation

B-Rep surfaces are tessellated to triangle meshes for rendering:

- **Parallel execution** via Rayon `par_iter` across faces
- **Adaptive refinement** based on surface curvature
- Target: **< 100 ms for 100K faces**
- Output: Vertex positions, normals, and UV coordinates

OpenCASCADE Fallback

When Truck lacks coverage (complex surface intersections, full IGES support), the system falls to `opencascade-rs` 0.2 via C++ FFI:

> ⚠ **Limitation**
>
> OpenCASCADE is **not WASM-compatible**. It is used only in desktop and cloud deployment modes. The WASM web build relies exclusively on Truck.

Sketch Operations System

The cad-kernel includes a comprehensive **sketch operations system** inspired by Blender's CAD_Sketcher addon, implementing all 14 Blender/CAD patterns:

Sketch Entities

Seven entity types are supported in 2D sketches:

Entity	Description	Constraint Support
Point	2D point in sketch plane	Coincident, Fixed, PointOnLine, PointOnCircle
Line	Infinite or bounded line segment	Horizontal, Vertical, Parallel, Perpendicular, Tangent
Circle	Circle defined by center + radius	Concentric, EqualRadius, Tangent
Arc	Circular arc (center, start angle, end angle)	Tangent, Concentric, EqualRadius
Ellipse	Ellipse with major/minor axes	Concentric
Spline	Cubic Bézier spline with control points	Tangent, PointOnLine
Construction	Reference geometry (not extruded)	All constraint types

Sketch Tools (StatefulTool Pattern)

The tool system uses a **Blender-inspired StatefulTool** lifecycle with `ToolStateMachine`:

Tool	States	Description
`LineTool`	Idle ⟶ First Point ⟶ Drawing	Draw line segments with snap-to-grid/entity
`CircleTool`	Idle ⟶ Center ⟶ Radius	Draw circles by center + radius point
`ArcTool`	Idle ⟶ Center ⟶ Start ⟶ End	Draw arcs by center + start/end angles
`RectangleTool`	Idle ⟶ Corner1 ⟶ Corner2	Draw axis-aligned rectangles (4 lines + constraints)
`SplineTool`	Idle ⟶ Points ⟶ Complete	Draw cubic Bézier splines with control handles

© 2026 — MIT OR Apache-2.0

Tool	States	Description
PointTool	Idle ⟶ Placing	Place construction/reference points
TrimTool	Idle ⟶ Selecting	Trim entities at intersection points

Snap Engine

The snap engine (snap.rs, 569 lines) supports 7 snap types for precision drawing:

- **Grid** — snap to configurable grid spacing
- **Endpoint** — snap to entity endpoints
- **Midpoint** — snap to entity midpoints
- **Center** — snap to circle/arc centers
- **Intersection** — snap to entity intersection points
- **Perpendicular** — snap perpendicular to entities
- **Tangent** — snap tangent to curves

The PickingColorMap provides GPU-accelerated entity picking by assigning unique colors to sketch entities.

Advanced Operations (sketch_ops.rs)

The sketch_ops.rs module (1,238 lines, 13 tests) implements advanced sketch operations:

- **Trim** — trim entities at intersections (line-line, line-circle, circle-circle)
- **Bevel** — create chamfer/fillet at entity intersections
- **Offset** — parallel offset of sketch profiles
- **Entity Walker** — traverse connected entity chains for profile detection
- **Bézier Utilities** — cubic Bézier evaluation, splitting, arc length
- **Intersection Engine** — compute entity-entity intersection points
- **Mesh Conversion** — convert sketch profiles to triangle meshes for preview

Snapshot System

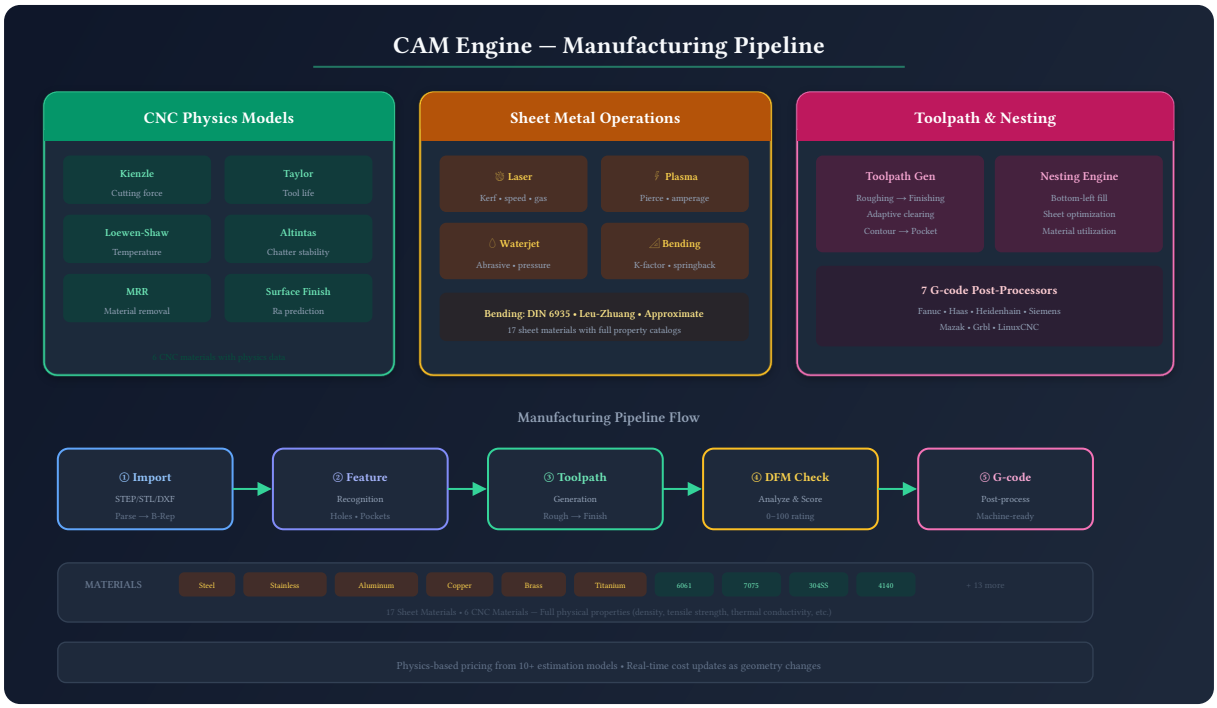
The snapshot.rs module provides sketch state management:

- **SketchSnapshot** — serializable snapshot of complete sketch state
- **ToolSnapshotManager** — manages tool state across undo/redo boundaries
- **ClipboardBuffer** — copy/paste sketch entities with constraint preservation

© 2026 — MIT OR Apache-2.0

CAM Engine

The CAM (Computer-Aided Manufacturing) engine integrates CNC machining physics, sheet metal fabrication, toolpath generation, and nesting optimization into a unified manufacturing pipeline.



CNC Physics Models

The CNC module implements **six physics-based models** for accurate machining simulation and cost estimation:

Cutting Force — Kienzle Model

The Kienzle equation predicts specific cutting force:

$$F_c = k_{c1.1} \cdot h^{1-m_c} \cdot b$$

where $k_{c1.1}$ is the specific cutting force for 1 mm² chip cross-section, h is chip thickness, m_c is the material exponent, and b is chip width.

Tool Life — Taylor Equation

Tool wear prediction uses the extended Taylor equation:

$$V_c \cdot T^n = C$$

where V_c is cutting velocity, T is tool life in minutes, and n, C are material-dependent constants.

Cutting Temperature — Loewen-Shaw

Thermal modeling for cutting zone temperature:

$$\theta = \theta_0 + \frac{F_c \cdot V_c}{A \cdot k} \cdot f(R)$$

Chatter Stability — Altintas Model

Stability lobe diagram computation to avoid self-excited vibration:

$$a_{\text{lim}} = -\frac{1}{2 \cdot K_f \cdot \text{Re}[G(j\omega_c)]}$$

Material Removal Rate

$$\text{MRR} = a_p \cdot a_e \cdot v_f$$

where a_p is depth of cut, a_e is width of cut, and v_f is feed rate.

Surface Finish Prediction

$$R_a = \frac{f^2}{32 \cdot r_\varepsilon}$$

where f is feed per revolution and r_ε is tool nose radius.

Sheet Metal Operations

Cutting Methods

Method	Parameters	Speed	Tolerance
🔥 Laser	Kerf width, gas type, pierce time, focal position	High	±0.1 mm
⚡ Plasma	Amperage, arc voltage, pierce height, cut height	Very High	±0.5 mm
💧 Waterjet	Pressure (MPa), abrasive flow, standoff distance	Medium	±0.05 mm
📐 Punching	Tool size, stroke rate, nibble spacing	High	±0.1 mm

Bending Models

Three bending formulas with springback prediction:

Formula	Description	Use Case
DIN 6935	$K = \left(\frac{\pi}{180}\right) \cdot \alpha \cdot (r_i + K_{\text{factor}} \cdot t)$ — standard bend allowance	General purpose

© 2026 — MIT OR Apache-2.0

Formula	Description	Use Case
Leu-Zhuang	Non-linear springback model for large-radius bends	High precision
Approximate	Simplified K-factor method for quick estimates	Quick quoting

Where K_{factor} ranges from 0.3 (soft materials, tight radii) to 0.5 (hard materials, large radii).

Material Catalogs

Sheet Materials (17)

Material	Density	Tensile	Conductivity	Max Thickness
Mild Steel	7.85 g/cm³	400 MPa	50 W/m·K	25 mm
Stainless 304	8.00 g/cm³	505 MPa	16 W/m·K	20 mm
Stainless 316	8.00 g/cm³	515 MPa	16 W/m·K	20 mm
Aluminum 5052	2.68 g/cm³	230 MPa	138 W/m·K	12 mm
Aluminum 6061	2.70 g/cm³	310 MPa	167 W/m·K	12 mm
Copper C110	8.94 g/cm³	220 MPa	388 W/m·K	6 mm
Brass C260	8.53 g/cm³	340 MPa	120 W/m·K	6 mm
Titanium Gr2	4.51 g/cm³	345 MPa	16 W/m·K	10 mm

Plus 9 additional materials: Galvanized, Spring Steel, Corten, Inconel 625, Monel 400, Zinc, Nickel 200, Hastelloy C276, Zircaloy

CNC Materials (6)

Material	$k_{c1.1}$	Taylor n	Hardness
Aluminum 6061-T6	800 N/mm²	0.25	95 HB
Aluminum 7075-T6	900 N/mm²	0.20	150 HB
Stainless 304	2200 N/mm²	0.20	187 HB
Steel 4140	2100 N/mm²	0.25	197 HB
Titanium Ti-6Al-4V	1800 N/mm²	0.15	334 HB
Brass C360	780 N/mm²	0.30	78 HB

© 2026 — MIT OR Apache-2.0

Toolpath Generation

The toolpath pipeline produces machine-ready G-code:

1. **Feature Recognition** — identify machinable features (holes, pockets, profiles)
2. **Operation Planning** — select tools, determine roughing/finishing strategy
3. **Toolpath Computation** — generate tool center-line paths with collision avoidance
4. **Post-Processing** — translate generic toolpath to machine-specific G-code

G-code Post-Processors

Eight machine controller formats are supported:

Fanuc | Haas | Heidenhain | Mazak
Grbl | LinuxCNC | Marlin | Klipper

Nesting Optimization

The nesting engine optimizes part placement on sheet stock to minimize material waste:

- **Algorithm**: Bottom-left fill with rotation (0°, 90°, 180°, 270°)
- **Target**: < 1 second for 100 parts
- **Metrics**: Material utilization percentage, waste area, nested count
- **Output**: Part placement coordinates + cutting path order

> **i Future Enhancement**
>
> Planned: genetic algorithm-based nesting for complex part geometries with irregular contours, targeting 85%+ material utilization for production runs.

© 2026 — MIT OR Apache-2.0

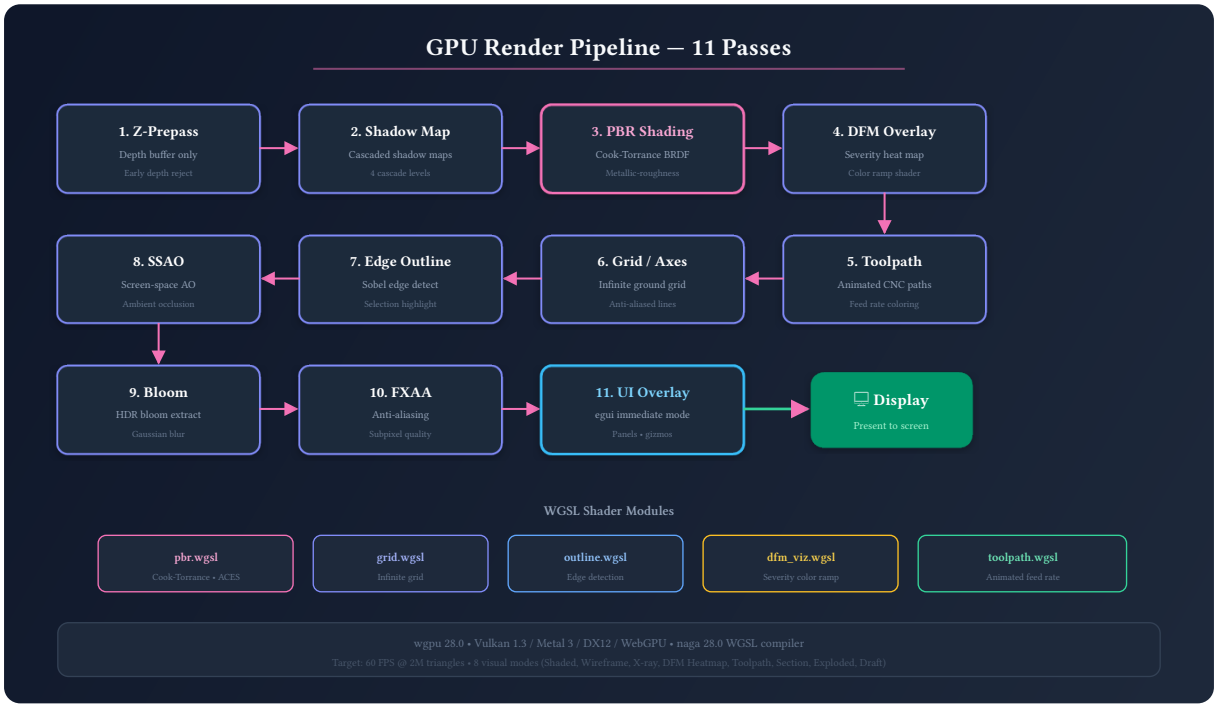
GPU Acceleration & Renderer

Current Implementation Status

> ⚠️ **Implementation Note**
>
> The renderer crate currently provides **foundational infrastructure** (6 files, 553 lines) built on Three.js via the React/Tauri frontend. The advanced wgpu-based GPU pipeline described in this chapter represents the **target architecture** for future phases. The current implementation handles basic scene management, camera control (orbit/pan/zoom), and mesh display through the Three.js/React Three Fiber stack.

Target Rendering Architecture

The r3ditor renderer is built on **wgpu 28**, providing a cross-platform GPU abstraction over Vulkan 1.3, Metal 3, DirectX 12, and WebGPU. The rendering pipeline consists of **11 passes** orchestrated by the render engine.



11-Pass Render Pipeline

Pass	Type	Description	Cost
1. Z-Prepass	Depth	Populate depth buffer for early-Z rejection in later passes	Low
2. Shadow Map	Depth	Cascaded shadow maps (4 levels) from directional light	Medium
3. PBR Shading	Color	Cook-Torrance BRDF with metallic-roughness workflow	High
4. DFM Overlay	Color	Severity heat map visualization from DFM analysis	Low
5. Toolpath	Color	Animated CNC toolpath with feed-rate color coding	Low
6. Grid / Axes	Color	Infinite ground grid with anti-aliased lines	Low
7. Edge Outline	Post	Sobel edge detection for selection highlighting	Medium
8. SSAO	Post	Screen-space ambient occlusion for depth perception	Medium
9. Bloom	Post	HDR bloom extraction + Gaussian blur composite	Low
10. FXAA	Post	Fast approximate anti-aliasing (subpixel quality)	Low
11. UI Overlay	Blit	egui immediate-mode UI rendering over the 3D viewport	Low

PBR Shading Model

The primary shading pass implements the **Cook-Torrance microfacet BRDF**:

$$f_r = \frac{D \cdot F \cdot G}{4 \cdot (\omega_o \cdot n) \cdot (\omega_i \cdot n)}$$

Where:
- D = GGX/Trowbridge-Reitz normal distribution function
- F = Fresnel-Schlick approximation: $F_0 + (1 - F_0)(1 - \cos\theta)^5$
- G = Smith's Geometry function with Schlick-GGX

The shader applies **ACES filmic tone mapping** for HDR to LDR conversion and outputs in sRGB color space.

WGSL Shader Modules

Five WGSL shader files power the rendering:

Shader	Lines	Purpose
pbr.wgsl	200	Cook-Torrance BRDF, point/directional lighting, ACES tone mapping, shadow sampling
grid.wgsl	80	Infinite ground grid with configurable scale, anti-aliased line rendering, depth fade

© 2026 — MIT OR Apache-2.0

Shader	Lines	Purpose
outline.wgsl	60	Sobel edge detection from depth/normal buffer, selection highlight with configurable color
dfm_viz.wgsl	70	Severity-to-color ramp (green → yellow → red), opacity blending, threshold filtering
toolpath.wgsl	90	Animated line rendering with time-based progress, feed rate to color mapping, depth testing

Shader Budget

Shader	ALU Ops	Texture Reads	Budget
PBR	120	4 (albedo, normal, metal-rough, shadow)	< 2 ms
Grid	30	0	< 0.5 ms
Outline	40	2 (depth, normal)	< 0.5 ms
DFM Viz	25	1 (severity buffer)	< 0.3 ms
Toolpath	35	0	< 0.3 ms

Visual Modes

The renderer supports **8 distinct visual modes**, switchable at runtime:

Shaded — Full PBR with shadows and AO

X-Ray — Semi-transparent solid with visible edges

Toolpath — Animated CNC path visualization

Exploded — Assembly parts separated along axes

Wireframe — Edge-only display with hidden-line removal

DFM Heatmap — Severity color overlay on geometry

Section View — Clipping plane with cross-section fill

Draft / Technical — Engineering drawing style (line art)

GPU Compute Strategy

Beyond rendering, wgpu compute shaders accelerate geometry operations:

Operation	Description	Speedup
Tessellation	NURBS surface → triangle mesh on GPU	3–5×
BVH Build	Bounding volume hierarchy for ray casting	2–4×
Mesh Simplification	Edge collapse with quadric error metrics	2–3×
Nesting	Part placement optimization (future)	5–10×

© 2026 — MIT OR Apache-2.0

Camera System

The viewport camera supports multiple interaction modes:

- **Orbit** — rotate around focus point (middle mouse / Alt+LMB)
- **Pan** — translate parallel to view plane (Shift+MMB)
- **Zoom** — dolly in/out (scroll wheel / Ctrl+MMB)
- **Standard Views** — Front, Back, Top, Bottom, Left, Right, Isometric (keyboard shortcuts)
- **Fit All** — zoom to encompass all geometry (Home key)

3-Point Lighting

The default scene uses a professional 3-point lighting setup:

Light	Position	Intensity	Color
Key	(5, 10, 5)	1.0	Warm white (6500K)
Fill	(-5, 5, 3)	0.4	Cool blue tint
Back	(0, 5, −5)	0.3	Neutral white

Uniform Buffer Layout

GPU uniform data is structured for efficient access:

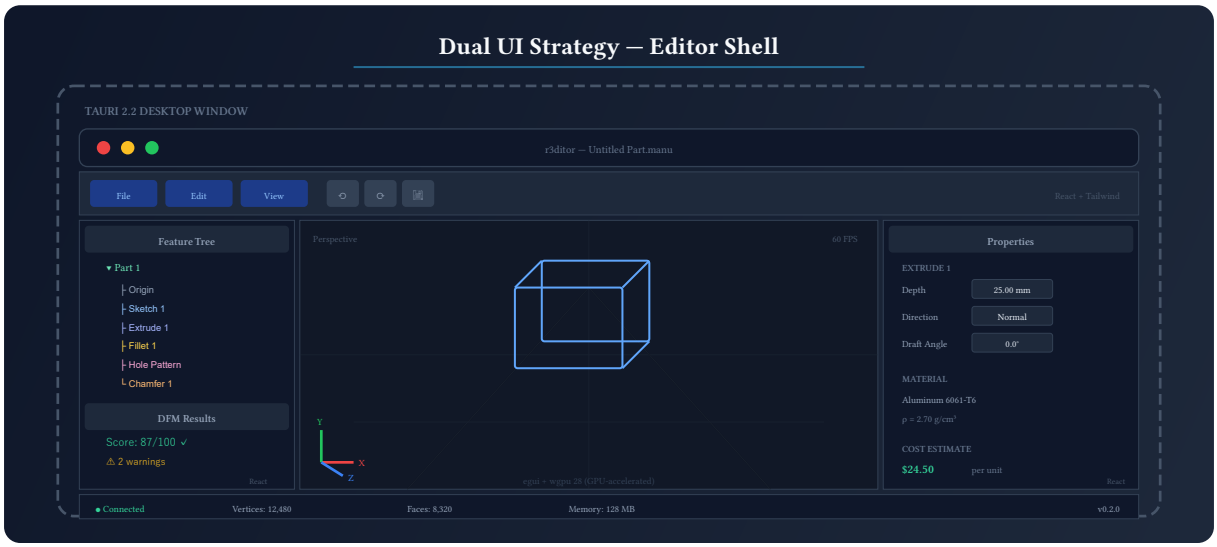
```
#[repr(C)]
pub struct SceneUniforms {
    pub view_projection: Mat4,  // Combined VP matrix
    pub camera_position: Vec4,  // Eye position
    pub light_direction: Vec4,  // Directional light
    pub light_color: Vec4,      // Light RGB + intensity
    pub time: f32,              // Elapsed seconds (animation)
    pub viewport_size: [f32; 2], // Width, height
}
```

© 2026 — MIT OR Apache-2.0

Editor Shell & UI

Dual UI Strategy

The r3ditor employs a **dual UI strategy** that combines the strengths of two rendering approaches:



Three.js — 3D Viewport

The 3D viewport is rendered via **Three.js 0.170** through React Three Fiber in the Tauri webview, providing GPU-accelerated 3D visualization.

- Mesh rendering with orbit/pan/zoom camera
- Entity picking via `PickingColorMap`
- Grid, axes, and sketch overlay rendering
- Integrated with React component lifecycle

React + Tailwind — Panels

All UI (toolbar, feature tree, properties panel, status bar, sketch toolbox) is built with **React 18.3 + TypeScript 5.6 + Tailwind CSS**, rendered in the Tauri webview.

- Rich component library with Tailwind styling
- Zustand 5.0 state management (1,062-line store)
- Responsive layout with CSS Grid
- Vite 6.4.1 hot-module replacement

Editor Shell Architecture

The `editor-shell` crate is the **ECS orchestrator** that wires all systems together:

Core Components

Module	Responsibility
app.rs	Main EditorApp struct — initializes all systems, manages tool activation/deactivation lifecycle, processes command queue
world.rs	World struct with sketches: HashMap, active_sketch, ToolSnapshotManager, ClipboardBuffer, SnapConfig, PickingColorMap
commands.rs	22 EditorCommand variants — all user operations as typed, undoable commands
tools.rs	21 tool types with StatefulTool lifecycle, ToolStateMachine, 7 sketch tools (Line, Circle, Arc, Rectangle, Spline, Point, Trim)
input.rs	Input handler — maps keyboard/mouse events to commands and tool state transitions
tauri_bridge	37 IPC command handlers in apps/desktop/src-tauri/src/commands.rs (692 lines)

EditorApp Lifecycle

```
pub struct EditorApp {
    world: World,                    // Sketch state, entities, constraints
    cad_kernel: CadKernel,
    cam_engine: CamEngine,
    constraint_solver: ConstraintSolver,
    dfm_analyzer: DfmAnalyzer,
    plugin_host: Option<PluginHost>,
    command_queue: VecDeque<EditorCommand>,
    undo_stack: Vec<EditorCommand>,
    redo_stack: Vec<EditorCommand>,
    active_tool: Option<Box<dyn StatefulTool>>,  // Blender-style tool lifecycle
    tool_snapshot_mgr: ToolSnapshotManager,
    clipboard: ClipboardBuffer,
}
```

The World struct manages sketch-specific state:

```
pub struct World {
    pub sketches: HashMap<Uuid, Sketch>,    // All sketches by ID
    pub active_sketch: Option<Uuid>,        // Currently edited sketch
    pub snap_config: SnapConfig,            // Grid/entity/midpoint snap settings
    pub picking_color_map: PickingColorMap,  // GPU color-based entity picking
}
```

The app processes commands and manages tool lifecycle:

1. activate_tool(tool_type) — instantiate StatefulTool, enter Idle state
2. handle_input() — route events to active tool state machine
3. process_commands() — dequeue and execute EditorCommands
4. solve_constraints() — resolve sketch constraints via 4-stage cascade
5. deactivate_tool() — finalize tool, commit or cancel operation

Command System

All user interactions are expressed as **typed commands**:

Command	Description	Undoable
`CreateFeature`	Add a new feature to the tree (extrude, fillet, etc.)	✓
`DeleteFeature`	Remove a feature and all dependents	✓
`ModifyFeature`	Change feature parameters (depth, radius, etc.)	✓
`SelectEntity`	Set the current selection (vertex, edge, face, body)	✗
`SetViewMode`	Switch visual mode (shaded, wireframe, x-ray)	✗
`SetMaterial`	Assign material to a body	✓
`ImportFile`	Load STEP/STL/DXF file	✓
`ExportFile`	Save to STEP/STL/DXF/G-code	✗
`Undo` / `Redo`	Navigate command history	—
`RunDfm`	Trigger DFM analysis	✗
`GenerateToolpath`	Compute CNC toolpath + G-code	✗
`EnterSketchMode`	Activate sketch editing on a plane/face	✗
`ExitSketchMode`	Commit sketch and return to 3D modeling	✗
`ActivateTool`	Switch active sketch tool (Line, Circle, Arc, etc.)	✗
`AddSketchEntity`	Add point/line/circle/arc/spline to active sketch	✓
`DeleteSketchEntity`	Remove entity from sketch (cascades constraints)	✓
`AddConstraint`	Apply constraint to sketch entities	✓
`RemoveConstraint`	Remove a sketch constraint	✓
`TrimEntity`	Trim sketch entity at intersection	✓
`CopyEntities`	Copy selected entities to clipboard	✗
`PasteEntities`	Paste clipboard entities into sketch	✓
`SetSnapConfig`	Configure snap types and grid spacing	✗

© 2026 — MIT OR Apache-2.0

Keyboard Shortcuts

Key	Modifier	Action
Ctrl+Z	—	Undo
Ctrl+Shift+Z	—	Redo
Ctrl+S	—	Save
Ctrl+O	—	Open file
Delete	—	Delete selection
F	—	Fit all to view
1	Numpad	Front view
3	Numpad	Right view
7	Numpad	Top view
5	Numpad	Toggle perspective/orthographic
W	—	Wireframe toggle
X	—	X-ray toggle

React Frontend

Component Architecture

Component	Description	State Source
App.tsx	Root layout — CSS Grid with sidebar/main/panel areas	Zustand store
Toolbar.tsx	Top toolbar with file ops, view controls, and action buttons	Zustand store
FeatureTree.tsx	Left panel — hierarchical feature list with drag-and-drop	Zustand store
PropertiesPanel.tsx	Right panel — selected feature parameters, material, cost	Zustand store
Viewport3D.tsx	Center — Three.js/R3F canvas with orbit controls, entity picking, sketch overlay	Zustand store
StatusBar.tsx	Bottom bar — connection status, vertex/face counts, memory, version	Zustand store

© 2026 — MIT OR Apache-2.0

State Management

The frontend uses **Zustand** for global state:

```
interface EditorStore {
  // 3D Model State
  features: Feature[];
  selectedId: string | null;
  viewMode: ViewMode;
  material: Material | null;
  dfmScore: number | null;
  // Sketch State
  activeSketchId: string | null;
  sketchEntities: SketchEntity[];
  sketchConstraints: Constraint[];
  activeTool: ToolType | null;
  snapConfig: SnapConfig;
  // Connection
  isConnected: boolean;
  // Actions (37 Tauri invoke wrappers)
  createFeature: (feature: Feature) => Promise<void>;
  enterSketchMode: (planeId: string) => Promise<void>;
  activateTool: (tool: ToolType) => void;
  addSketchEntity: (entity: SketchEntity) => Promise<void>;
  addConstraint: (constraint: Constraint) => Promise<void>;
  solveConstraints: () => Promise<SolveResult>;
  // ... 30+ more actions
}
```

Tauri IPC Bridge

Communication between React and Rust uses Tauri's typed IPC:

```
// Frontend ⟶ Rust (37 registered IPC handlers)
await invoke('import_file', { path: filePath });
await invoke('create_feature', { feature: newFeature });
await invoke('enter_sketch_mode', { planeId, sketchId });
await invoke('add_sketch_entity', { sketchId, entity });
await invoke('add_constraint', { sketchId, constraint });
await invoke('solve_constraints', { sketchId });
await invoke('activate_tool', { toolType: 'line' });
const estimate = await invoke('estimate_cost', { material, geometry });

// Rust ⟶ Frontend (events)
listen('feature-tree-updated', (event) => { ... });
listen('sketch-updated', (event) => { ... });
listen('constraint-solved', (event) => { ... });
```

Technology Stack

- React 18.3
- TypeScript 5.6
- Tailwind CSS
- Zustand 5.0
- Vite 6.4.1
- Three.js 0.170

© 2026 — MIT OR Apache-2.0

API Gateway & Cloud Services

API Gateway

The `api-gateway` crate provides a **RESTful API** built on Axum 0.7 with JWT authentication, file upload handling, and Server-Sent Events for real-time job status updates.

Route Map

Method	Path	Description	Auth
`GET`	`/api/health`	Health check — returns service status, version, uptime	None
`POST`	`/api/uploads`	Upload CAD file (multipart) → store in MinIO → return upload ID	JWT
`GET`	`/api/uploads/:id`	Retrieve upload metadata and download URL	JWT
`POST`	`/api/jobs`	Create analysis job → push to Redis stream → return job ID	JWT
`GET`	`/api/jobs/:id`	Get job status and results	JWT
`GET`	`/api/jobs/:id/stream`	SSE stream for real-time job progress updates	JWT
`GET`	`/api/materials`	List all materials (sheet + CNC) with properties	Optional
`GET`	`/api/materials/:id`	Get detailed material properties	Optional
`POST`	`/api/quotes`	Generate manufacturing quote from job results	JWT
`GET`	`/api/quotes/:id`	Retrieve generated quote	JWT

Authentication

JWT-based authentication with configurable secret:

- **Token format**: Bearer token in Authorization header
- **Claims**: `sub` (tenant ID), `exp` (expiration), `iat` (issued at)
- **Middleware**: Tower layer applied to protected routes
- **Tenant isolation**: All queries scoped by `tenant_id` from JWT claims

© 2026 — MIT OR Apache-2.0

Application State

```
pub struct AppState {
    pub db: PgPool,          // PostgreSQL connection pool
    pub redis: redis::Client, // Redis for job queues
    pub s3: aws_sdk_s3::Client, // MinIO S3-compatible storage
    pub jwt_secret: String,    // JWT signing key
}
```

Worker Services

Three stateless worker services consume jobs from Redis streams:

Worker Architecture

Worker	Function	Input	Output
`worker-cad`	Import CAD files (STEP/STL/DXF) → parse → extract geometry → store mesh	File bytes	Mesh + metadata
`worker-analysis`	Run DFM analysis → score geometry → find manufacturing issues	Mesh + material	DFM findings + score
`worker-cam`	Generate toolpaths → optimize nesting → produce G-code	Mesh + material + config	Toolpath + G-code

All workers follow the same pattern:

1. **Read** from Redis stream (`XREADGROUP`)
2. **Download** file from MinIO
3. **Process** using core crates (cad-kernel / dfm-analyzer / cam-engine)
4. **Store** results in PostgreSQL
5. **Acknowledge** message (`XACK`)
6. **Publish** status update (triggers SSE to client)

Redis Stream Protocol

```
Stream: jobs:pending
Consumer Group: workers
Message Format:
{
  "job_id": "uuid",
  "upload_id": "uuid",
  "tenant_id": "uuid",
  "job_type": "analysis|cad_import|cam_generate",
```

```
  "parameters": { ... }
}
```

Database Schema

PostgreSQL 16 with 4 core tables:

Table	Description	Key Indexes
`uploads`	File metadata: name, hash, size, S3 key, tenant_id, timestamps	PK: `id`
`jobs`	Job status: upload_id, tenant_id, type, status, result_json, timestamps	`idx_jobs_upload_id`, `idx_jobs_tenant_id`
`dfm_findings`	DFM analysis results: job_id, rule_id, severity, description, location	`idx_findings_job_id`
`quotes`	Generated quotes: job_id, tenant_id, line items, total, currency	`idx_quotes_job_id`

Object Storage

MinIO (S3-compatible) stores binary assets:

Bucket	Contents
`cad-uploads`	Original uploaded CAD files (STEP, STL, DXF, 3MF)
`meshes`	Processed triangle meshes (binary format)
`gcode`	Generated G-code files
`thumbnails`	Rendered preview images

API Documentation

The API is documented via **OpenAPI 3.0** using `utoipa`:

- Interactive Swagger UI at `/swagger-ui`
- OpenAPI JSON spec at `/api-doc/openapi.json`
- All types annotated with `#[derive(ToSchema)]`
- Request/response examples for every endpoint

© 2026 — MIT OR Apache-2.0

Error Handling

Consistent error responses across all endpoints:

```
{
  "error": {
    "code": "UPLOAD_TOO_LARGE",
    "message": "File size exceeds 100 MB limit",
    "details": { "max_bytes": 104857600, "actual_bytes": 157286400 }
  }
}
```

Error types:

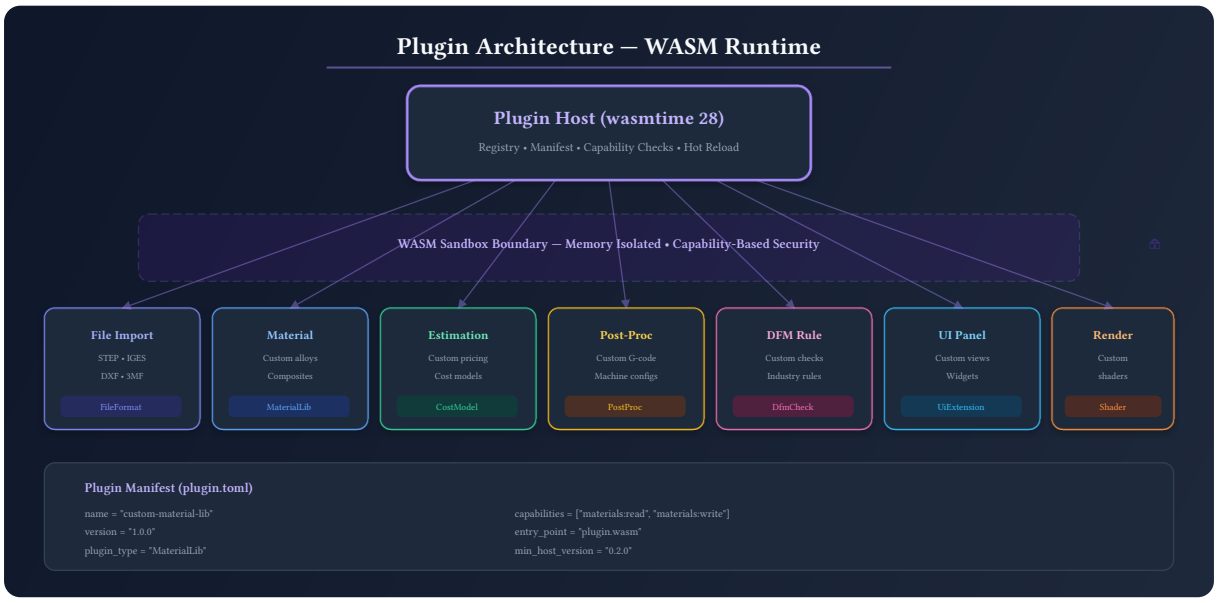
- `400` — Validation errors (bad request body, invalid parameters)
- `401` — Authentication required (missing/expired JWT)
- `403` — Authorization denied (wrong tenant)
- `404` — Resource not found
- `413` — Payload too large
- `500` — Internal server error

© 2026 — MIT OR Apache-2.0

Plugin System

Architecture

The r3ditor plugin system is built on **wasmtime 28**, providing a sandboxed WASM runtime for third-party extensions. Plugins are isolated from the host and from each other, with access controlled by a **capability-based security model**.



Plugin Categories

Seven plugin categories are supported, each with a dedicated host interface:

Category	Interface Trait	Description	Capabilities
File Format	`FileFormat`	Import/export custom file formats (IGES, 3MF, Parasolid, etc.)	`fs:read`, `fs:write`
Material Library	`MaterialLib`	Add custom material definitions with physical properties	`materials:read`, `materials:write`
Cost Estimation	`CostModel`	Custom pricing models (vendor-specific, volume discounts)	`estimation:read`, `estimation:write`

Category	Interface Trait	Description	Capabilities
Post-Processor	PostProc	Custom G-code post-processors for proprietary machine controllers	gcode:read, gcode:write
DFM Rule	DfmCheck	Custom design-for-manufacturability rules for industry standards	geometry:read, dfm:write
UI Extension	UiExtension	Custom panels, widgets, and toolbar buttons in the editor	ui:read, ui:write
Render Shader	Shader	Custom WGSL shaders for specialized visualization	render:write

Plugin Manifest

Every plugin is described by a plugin.toml manifest:

```toml
[plugin]
name = "custom-material-lib"
version = "1.0.0"
description = "Aerospace alloy material library with Ti-6Al-4V variants"
author = "r3ditor Community"
license = "MIT"
plugin_type = "MaterialLib"
entry_point = "plugin.wasm"
min_host_version = "0.2.0"

[capabilities]
required = ["materials:read", "materials:write"]
optional = ["fs:read"]

[metadata]
homepage = "https://github.com/example/aerospace-materials"
repository = "https://github.com/example/aerospace-materials"
keywords = ["aerospace", "titanium", "materials"]
```

Security Model

Capability-Based Access

Plugins request capabilities in their manifest; the host grants or denies each capability at load time:

Capability	Description	Risk Level
fs:read	Read files from the project directory (not system files)	Medium

Capability	Description	Risk Level
`fs:write`	Write files to the project directory	Medium
`materials:read`	Read from the material catalog	Low
`materials:write`	Add/modify materials in the catalog	Low
`geometry:read`	Read the B-Rep model and mesh data	Low
`geometry:write`	Modify geometry (add features, boolean ops)	High
`estimation:read`	Read cost estimates	Low
`estimation:write`	Modify cost estimates	Medium
`dfm:write`	Add DFM findings/rules	Low
`gcode:read`	Read generated G-code	Low
`gcode:write`	Modify or generate G-code	High
`ui:read`	Read UI state (selection, view mode)	Low
`ui:write`	Add panels, buttons, and overlays	Medium
`render:write`	Register custom WGSL shaders	Medium
`network`	Make outbound HTTP requests (for licensing, data fetch)	High

Sandbox Guarantees

- **Memory isolation** — each WASM module has its own linear memory, no shared state
- **No raw system calls** — all I/O goes through capability-checked host functions
- **Resource limits** — configurable memory cap (default 64 MB per plugin)
- **Timeout** — plugin functions abort after 5 seconds (configurable)
- **No file system access** unless `fs:read`/`fs:write` capability is granted

Plugin Host API

The host exposes a set of functions to plugins via the WASM import mechanism:

```
// Host functions available to plugins (via wasmtime linker)
pub trait PluginHost {
    fn log(&self, level: LogLevel, message: &str);
    fn get_material(&self, id: &str) -> Option<Material>;
    fn set_material(&mut self, material: Material) -> Result<()>;
    fn get_geometry(&self) -> Option<MeshData>;
    fn add_dfm_finding(&mut self, finding: DfmFinding);
```

```
    fn read_file(&self, path: &str) -> Result<Vec<u8>>;
    fn write_file(&mut self, path: &str, data: &[u8]) -> Result<()>;
}
```

Registry & Discovery

Plugins are managed through a **local registry**:

- **Installation**: Copy `.wasm` + `plugin.toml` to `~/.r3ditor/plugins/<name>/`
- **Discovery**: Host scans plugin directory at startup, validates manifests
- **Loading**: WASM modules compiled to native code (wasmtime AOT)
- **Hot Reload**: File watcher detects changes → unload → recompile → reload
- **Conflict Resolution**: Plugin load order determined by dependency graph; conflicts are reported

> **i Plugin Development**
>
> Plugins can be written in any language that compiles to WASM: Rust (primary), C/C++, AssemblyScript, Go (via TinyGo), or Zig. The plugin SDK provides Rust proc-macros for ergonomic development.

Example: Custom Material Plugin

```
use r3ditor_plugin_sdk::prelude::*;

#[plugin(name = "aerospace-ti64", version = "1.0.0")]
pub struct AerospaceTi64;

impl MaterialLib for AerospaceTi64 {
    fn materials(&self) -> Vec<Material> {
        vec![Material {
            name: "Ti-6Al-4V (Annealed)".into(),
            density: 4.43,        // g/cm³
            tensile_strength: 950.0, // MPa
            thermal_conductivity: 6.7, // W/m·K
            hardness: 334.0,      // HB
            specific_cutting_force: 1800.0, // N/mm²
            taylor_n: 0.15,
        }]
    }
}
```

Data Formats & File I/O

Native Format: `.manu`

The r3ditor native project file uses the `.manu` extension — a **ZIP archive** containing structured JSON metadata and binary mesh payloads:

```
project.manu (ZIP archive)
├── manifest.json       # Version, metadata, feature list
├── geometry/
│   ├── brep.json       # B-Rep topology (Vertex, Edge, Face, Shell, Solid)
│   ├── mesh.bin        # Binary tessellated mesh (position + normal + UV)
│   └── features.json   # Parametric feature history stack
├── manufacturing/
│   ├── toolpaths.json  # Generated toolpath segments
│   ├── gcode/          # Post-processed G-code files per machine
│   └── nesting.json    # Sheet nesting layout
├── materials/
│   └── catalog.json    # Materials used in project
├── analysis/
│   ├── dfm.json        # DFM findings and severity scores
│   └── cost.json       # Cost estimation breakdown
├── assets/
│   └── thumbnails/     # Preview thumbnails (PNG 256×256)
└── plugins/
    └── state.json      # Per-plugin saved state
```

Binary Mesh Format

The `mesh.bin` file uses a compact binary layout optimized for GPU upload:

Section	Offset	Type	Description
Magic	0	`u8 × 4`	`0x4D 0x41 0x4E 0x55` ("MANU")
Version	4	`u32`	Binary format version (currently 1)
Vertex Count	8	`u32`	Number of vertices
Index Count	12	`u32`	Number of triangle indices
Vertex Data	16	`f32 × 8 × N`	Per-vertex: pos.xyz + normal.xyz + uv.xy
Index Data	16 + 32N	`u32 × M`	Triangle indices (3 per triangle)

© 2026 — MIT OR Apache-2.0

Section	Offset	Type	Description
Checksum	tail	`u32`	CRC32 of all preceding bytes

Import Formats

R3ditor supports a comprehensive set of industry-standard import formats:

Format	Extension	Crate / Library	Notes
STEP AP203/AP214	`.step`, `.stp`	`truck-stepio`	Primary B-Rep interchange; preserves topology, assemblies, PMI
IGES	`.iges`, `.igs`	OpenCascade FFI	Legacy B-Rep format; trimmed surfaces, wireframes
STL (Binary)	`.stl`	`wasm-meshkit`	Triangle mesh only; auto-detect binary via header magic
STL (ASCII)	`.stl`	`wasm-meshkit`	Slower parsing; fallback if binary detection fails
OBJ	`.obj`	`tobj` crate	Mesh + materials (MTL); groups → bodies
3MF	`.3mf`	`threemf` crate	Additive manufacturing format; mesh + color + materials
DXF	`.dxf`	`dxf` crate	2D/3D wireframe; used for sheet metal flat patterns
glTF 2.0	`.gltf`, `.glb`	`gltf` crate	Scene transfer; mesh + PBR materials + transforms
Parasolid	`.x_t`, `.x_b`	Plugin	Via plugin system (commercial license required)

i Format Auto-Detection

The importer uses a two-pass detection strategy:
1. **Extension-based** — fast path using file extension
2. **Magic-byte sniffing** — fallback for misnamed files (e.g., binary STL with wrong extension)

Export Formats

Format	Extension	Crate / Library	Notes
STEP AP203	`.step`	`truck-stepio`	Full B-Rep with assemblies and metadata

© 2026 — MIT OR Apache-2.0

Format	Extension	Crate / Library	Notes
STL (Binary)	`.stl`	Custom writer	Compact binary output; 50 bytes per triangle
STL (ASCII)	`.stl`	Custom writer	Human-readable; larger file size
DXF	`.dxf`	`dxf` crate	2D flat pattern export for laser/waterjet cutting
G-code	`.nc`, `.gcode`	`r3ditor-cam`	Machine-specific via post-processors
3MF	`.3mf`	`threemf` crate	Color mesh export for 3D printing
PDF Report	`.pdf`	`printpdf` crate	DFM analysis report with annotated views
SVG	`.svg`	`svg` crate	2D drawing export (flat patterns, sections)

G-Code Post-Processors

Eight built-in post-processors generate machine-specific G-code:

Post-Processor	Target	Features
Fanuc	Fanuc 0i/30i CNC	Standard G/M codes, canned cycles G81–G89, macro B
Haas	Haas VF/ST series	Macros, visual programming, tool setting cycles
Mazak	Mazatrol Matrix	Conversational format + EIA/ISO G-code hybrid
Heidenhain	Heidenhain TNC	Conversational programming, cycle definitions, FK programming
Grbl	Open-source CNC	Reduced instruction set for hobby/desktop mills
LinuxCNC	LinuxCNC	O-word subroutines, named parameters, remapping
Marlin	3D Printer / CNC	G-code for FDM printers and simple CNC routers
Klipper	Klipper firmware	Extended G-code with macro support, pressure advance

File I/O Pipeline

Import Pipeline

```
File ⟶ Extension Detection ⟶ Magic-Byte Sniff ⟶ Format Router
  ⟶ STEP Parser (truck-stepio) ⟶ B-Rep Model
  ⟶ STL Parser (wasm-meshkit) ⟶ Mesh ⟶ Auto-heal (manifold check)
  ⟶ DXF Parser (dxf crate) ⟶ 2D Wireframe ⟶ Extrude to 3D
  ⟶ OBJ/glTF Parser ⟶ Mesh + Materials
⟶ Validation ⟶ Feature Recognition ⟶ Insert into ECS World
```

© 2026 — MIT OR Apache-2.0

Export Pipeline

```
ECS World ⟶ Format Selector
  ⟶ STEP Writer ⟶ B-Rep serialization ⟶ .step file
  ⟶ STL Writer ⟶ Tessellate ⟶ Binary/ASCII ⟶ .stl file
  ⟶ G-code Writer ⟶ Toolpath ⟶ Post-processor ⟶ .nc file
  ⟶ PDF Writer ⟶ Render views ⟶ Annotate ⟶ .pdf report
⟶ Compression (optional) ⟶ Output File
```

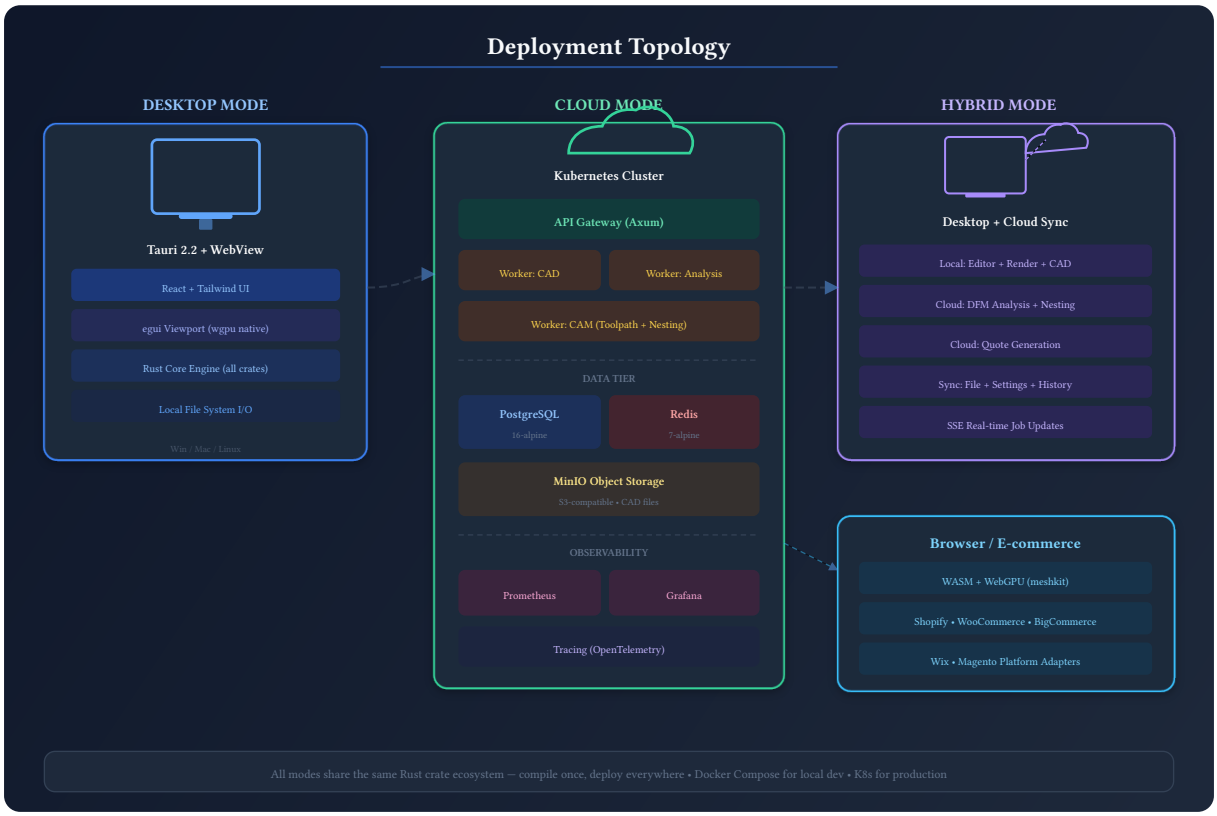
⚠ STEP Round-Trip Fidelity

STEP is the only format that preserves full B-Rep topology and parametric feature history on round-trip. All mesh-based formats (STL, OBJ, glTF, 3MF) lose parametric information and cannot be re-edited as features.

Deployment & Distribution

Deployment Modes

R3ditor supports three deployment topologies, each optimized for different use cases:



Mode	Target	Description
Desktop	Engineers & Designers	Full standalone application via Tauri 2.2. All computation runs locally. No internet required after activation. Best for proprietary designs.
Cloud	E-commerce & SaaS	Headless API server (Docker / Kubernetes). Web frontend connects via REST + SSE. Scales horizontally with Redis job queue.
Hybrid	Enterprise Teams	Desktop app for CAD modeling; cloud for heavy analysis, nesting, and collaboration. Sync via API.

Desktop Distribution

Tauri 2.2 Packaging

Platform	Installer	Size (est.)
Windows 10/11	`.msi` + `.exe` (NSIS)	45 MB
macOS 12+	`.dmg` + `.app` bundle	40 MB
Linux (Ubuntu 22+)	`.deb` + `.AppImage`	42 MB

Auto-Update

Tauri's built-in updater with signed update manifests:

```
{
  "endpoints": ["https://releases.r3ditor.io/{{target}}/{{current_version}}"],
  "dialog": true,
  "pubkey": "dW50cnVzdGVkIGNvbW1lbnQ..."
}
```

- **Differential updates** — only changed binary segments are downloaded
- **Rollback** — previous version preserved; automatic rollback on crash-on-start
- **Channel selection** — Stable / Beta / Nightly channels

Cloud Deployment

Docker Compose (Development / Small Teams)

Nine services compose the full cloud stack:

Service	Image	Port	Role
`api-gateway`	`r3ditor/api:latest`	3001	Axum REST API + SSE endpoint
`worker-cad`	`r3ditor/worker-cad:latest`	—	Geometry operations (B-Rep, boolean, tessellation)
`worker-analysis`	`r3ditor/worker-analysis:latest`	—	DFM checks, cost estimation, quoting
`worker-cam`	`r3ditor/worker-cam:latest`	—	Toolpath generation, nesting, G-code post
`postgres`	`postgres:16-alpine`	5432	Primary database (users, projects, materials)
`redis`	`redis:7-alpine`	6379	Job queue (Redis Streams) + session cache

© 2026 — MIT OR Apache-2.0

Service	Image	Port	Role
minio	minio/minio:latest	9000	S3-compatible object storage for files
jaeger	jaegertracing/all-in-one	16686	Distributed tracing (OpenTelemetry)
prometheus	prom/prometheus	9090	Metrics collection and alerting

Docker Compose File

```yaml
version: "3.9"
services:
  api-gateway:
    image: r3ditor/api:latest
    ports: ["3001:3001"]
    environment:
      DATABASE_URL: postgres://r3ditor:r3ditor@postgres/r3ditor
      REDIS_URL: redis://redis:6379
      S3_ENDPOINT: http://minio:9000
    depends_on: [postgres, redis, minio]
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:3001/api/v1/health"]
      interval: 10s

  worker-cad:
    image: r3ditor/worker-cad:latest
    environment:
      REDIS_URL: redis://redis:6379
      QUEUE: cad
    deploy:
      replicas: 2

  worker-analysis:
    image: r3ditor/worker-analysis:latest
    environment:
      REDIS_URL: redis://redis:6379
      QUEUE: analysis

  worker-cam:
    image: r3ditor/worker-cam:latest
    environment:
      REDIS_URL: redis://redis:6379
      QUEUE: cam

  postgres:
    image: postgres:16-alpine
    volumes: ["pgdata:/var/lib/postgresql/data"]
    environment:
      POSTGRES_DB: r3ditor
      POSTGRES_USER: r3ditor
      POSTGRES_PASSWORD: r3ditor

  redis:
    image: redis:7-alpine
    volumes: ["redisdata:/data"]
```

© 2026 — MIT OR Apache-2.0

```yaml
  minio:
    image: minio/minio:latest
    command: server /data --console-address ":9001"
    volumes: ["miniodata:/data"]

volumes:
  pgdata:
  redisdata:
  miniodata:
```

Kubernetes (Production)

For production deployments, r3ditor provides Helm charts:

```
helm repo add r3ditor https://charts.r3ditor.io
helm install r3ditor r3ditor/r3ditor-stack \
  --set api.replicas=3 \
  --set worker.cad.replicas=4 \
  --set worker.analysis.replicas=2 \
  --set worker.cam.replicas=2 \
  --set postgres.storageClass=gp3 \
  --set ingress.host=api.r3ditor.example.com
```

Horizontal Scaling Strategy

Component	Scaling Axis	Strategy
API Gateway	Horizontal	Stateless; add replicas behind load balancer. Target: <100ms p99.
Worker CAD	Horizontal	CPU-bound; scale by pending queue depth. Each replica handles 1 job.
Worker Analysis	Horizontal	Memory-bound (DFM models); scale by memory pressure.
Worker CAM	Horizontal	CPU-bound (nesting is $O(n^2)$); scale by job backlog.
PostgreSQL	Vertical + Read Replicas	Primary for writes; read replicas for queries. PgBouncer for connection pooling.
Redis	Vertical	Single node with AOF persistence. Redis Cluster for >100K jobs/min.
MinIO	Horizontal	Erasure coding across nodes for durability. CDN for file delivery.

Platform Adapters

R3ditor includes e-commerce platform adapters for embedded quoting:

Platform	Integration	Status	Method
Shopify	Storefront API + App Bridge	Production	Embedded iframe + webhook for order sync
WooCommerce	REST API v3 + Webhook	Production	WordPress plugin + React widget embed
BigCommerce	Storefront API + Webhooks	Beta	Custom widget via Script API injection
Magento 2	REST API + GraphQL	Planned	Module with admin panel integration
Wix	Velo API + Blocks	Planned	Wix Blocks app with custom element

i Monitoring & Observability

All services emit **OpenTelemetry traces** and **Prometheus metrics**. The default stack includes:

- **Jaeger** — distributed trace visualization
- **Prometheus** — metrics collection (request latency, queue depth, error rate)
- **Grafana** — dashboards (add `grafana/grafana` service for production)
- **Structured logging** — JSON logs via `tracing` crate with `tracing-opentelemetry` export

Performance Engineering

Performance Targets

R3ditor is engineered to meet strict latency and throughput targets across all subsystems:

Geometry Operations

Operation	Target	Budget	Strategy
Tessellation (100K faces)	< 16 ms	1 frame	Parallel tessellation via Rayon; adaptive LOD reduces face count
Boolean union/subtract	< 100 ms	6 frames	Truck B-Rep kernel with spatial hash acceleration
Fillet/chamfer	< 50 ms	3 frames	Edge subdivision on GPU; fallback to CPU for complex topology
Feature recognition	< 200 ms	12 frames	Pattern matching on B-Rep graph; cached results in ECS component
Constraint solving	< 10 ms	< 1 frame	Newton-Raphson with Jacobian caching; max 50 iterations
STL import (1M tri)	< 500 ms	30 frames	`wasm-meshkit` parallel parser; memory-mapped file I/O

Operation	Target	Budget	Strategy
STEP import (medium)	< 2 s	—	Streaming parser; UI remains responsive via async task

Rendering

Metric	Target	GPU Tier	Strategy
Frame rate (1M triangles)	60 fps	Mid-range	Frustum culling + LOD + instanced draw calls
Frame rate (5M triangles)	30 fps	Mid-range	Aggressive LOD; GPU occlusion query
Ray pick latency	< 5 ms	Any	GPU compute pick pass; single-pixel render target
Shader compile	< 200 ms	Any	WGSL pre-compilation at startup; pipeline cache
Texture upload (4K)	< 2 ms	Any	Staging buffer + async queue submit
View mode switch	< 16 ms	Any	Hot-swap pipeline; pre-built pipeline variants

Manufacturing

Operation	Target	Budget	Strategy
DFM analysis (full)	< 500 ms	—	Parallel rule evaluation via Rayon; early-exit on critical findings
Cost estimation	< 200 ms	—	Cached material lookup; pre-computed machine rates
Tool estimation	< 200 ms	—	Kienzle model with tabulated specific cutting force
Nesting (20 parts)	< 2 s	—	Bottom-left fill + Rayon parallel rotation search
G-code generation	< 1 s	—	Streaming writer; post-processor runs in parallel

Memory Budget

Pool	Budget	Management Strategy
Geometry (B-Rep + mesh)	512 MB	Arena allocator; LRU eviction for LOD levels; memory-mapped for large models
GPU buffers	256 MB	Staging ring buffer (16 MB) + device-local allocations; shared vertex/index pools
Undo/Redo stack	128 MB	Circular buffer of delta snapshots; old entries evicted FIFO
Texture atlas	64 MB	Shared material textures; mip chain generation on demand

© 2026 — MIT OR Apache-2.0

Pool	Budget	Management Strategy
Plugin WASM heap	64 MB / plugin	Configurable per-plugin; wasmtime memory limiter
File I/O buffers	32 MB	Memory-mapped files for import; streaming write for export
ECS components	64 MB	Archetype storage; component arrays pre-allocated by entity count estimate

Total desktop budget: 1.2 GB (comfortable on 4 GB+ systems with headroom for OS and browser)

Parallelism Architecture

Thread Budget

Pool	Threads	Runtime	Workload
Rayon global	N-1 (N = CPU cores)	Rayon	Parallel geometry, tessellation, nesting, DFM rule evaluation
Tokio runtime	4	Tokio	Async I/O: HTTP handlers, file I/O, database queries, SSE streams
Render thread	1	Dedicated	wgpu command encoding + queue submit; never blocked by compute
UI thread	1	Main	egui/React event loop; 16 ms budget per frame
File watcher	1	Dedicated	Plugin hot-reload, project file sync

Rayon Work-Stealing

Geometry operations leverage Rayon's work-stealing thread pool:

```rust
use rayon::prelude::*;

// Parallel tessellation of B-Rep faces
let meshes: Vec<Mesh> = solid.faces()
    .par_iter()
    .map(|face| tessellate_face(face, tolerance))
    .collect();

// Parallel DFM rule evaluation
let findings: Vec<DfmFinding> = rules
    .par_iter()
    .flat_map(|rule| rule.evaluate(&model))
    .collect();
```

Tokio Async I/O

All network and file I/O uses Tokio's async runtime:

```
// Concurrent file upload + database write
let (upload_result, db_result) = tokio::join!(
    minio.put_object(&bucket, &key, data),
    sqlx::query!("INSERT INTO projects ...")
        .execute(&pool)
);
```

Benchmarking

R3ditor includes a dedicated benchmark suite in `packages/bench/`:

Benchmark	Framework	What It Measures
`tessellation`	Criterion.rs	Tessellation throughput: faces/sec at various tolerance levels
`boolean_ops`	Criterion.rs	Union/subtract/intersect latency on parameterized model sizes
`dfm_analysis`	Criterion.rs	Full DFM pass on reference parts (thin wall, undercut, draft)
`nesting`	Criterion.rs	Nesting 5/10/20/50 parts on standard sheet sizes
`stl_import`	Criterion.rs	STL parse throughput: MB/s for binary and ASCII
`rendering`	Custom	Frame time histogram over 1000 frames at various triangle counts

Run benchmarks:

```
cargo bench -p r3ditor-bench
```

Results are output as HTML reports in `target/criterion/` with statistical analysis (mean, median, std dev, throughput).

> ⚠ **Performance Regression CI**
>
> The CI pipeline runs benchmarks on every merge to `main` and compares against the baseline. A regression > 10% on any critical path triggers a **pipeline failure** and blocks the merge.

Profiling Toolkit

Tool	Target	Usage
`tracy`	CPU + GPU	Frame profiler with timeline; `#[tracy::instrument]` on hot paths

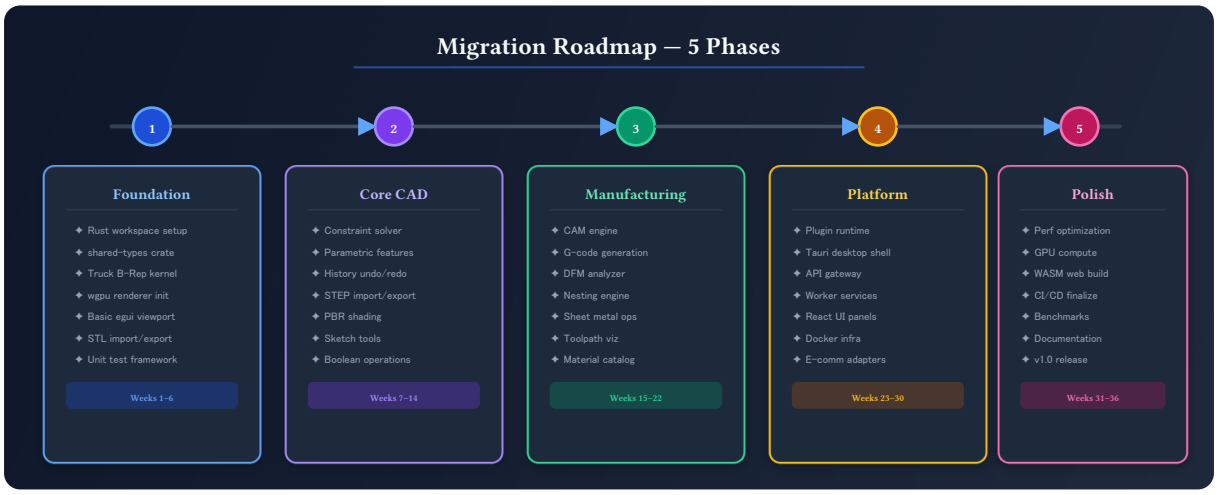
Tool	Target	Usage
`puffin`	CPU	In-app profiler overlay (egui integration); zero-cost when disabled
`wgpu` timestamps	GPU	Per-pass timing via timestamp queries; reported in debug overlay
Tokio Console	Async	Runtime inspector for task count, poll times, and waker stats
`cargo flamegraph`	CPU	Sampling profiler for full-stack flame graphs
`heaptrack`	Memory	Allocation tracking for memory leak detection

© 2026 — MIT OR Apache-2.0

Migration Path

Overview

R3ditor follows a **five-phase migration strategy** to transition from a legacy DFM analysis suite to a fully integrated CAD/CAM/Editor platform. Each phase is self-contained and delivers production-ready functionality.



Phase 1: Foundation Sprint (Weeks 1–4)

✓ Phase Complete

All foundation milestones delivered. Cargo workspace with 15 crates, Tauri shell, CI pipeline, and database migrations operational.

Goal: Establish the Rust workspace, build system, CI/CD pipeline, and core data structures.

Milestone	Deliverables	Week
M1.1	Cargo workspace with 15 crate stubs; `cargo check` passes	1
M1.2	ECS world (`hecs`) with basic entity/component model	1
M1.3	wgpu 28 renderer: window creation, clear color, triangle demo	2
M1.4	React + Vite frontend scaffold with Tailwind; Tauri shell boots	2
M1.5	CI pipeline: `cargo clippy`, `cargo test`, `cargo fmt --check`	3

Milestone	Deliverables	Week
M1.6	Docker Compose with PostgreSQL + Redis + MinIO; health check endpoint	3
M1.7	Database migrations (SQLx); seed data for materials catalog	4
M1.8	Integration test harness; code coverage reporting	4

Exit Criteria: `cargo build --workspace` succeeds; Tauri app launches with blank viewport; API responds to `/health`.

Phase 2: Geometry Engine (Weeks 5–10)

✓ Phase Complete

CAD kernel operational with 20 feature types, 4-stage constraint solver (19 types), full sketch system with 7 entity types, STL/STEP import/export, and undo/redo history. 125 kernel tests, 30 solver tests passing.

Goal: Implement the dual B-Rep kernel, parametric features, constraint solver, and mesh import/export.

Milestone	Deliverables	Week
M2.1	Truck B-Rep integration: create box, cylinder, sphere primitives	5
M2.2	Boolean operations (union, subtract, intersect) via Truck	6
M2.3	Parametric feature stack: 20 feature types (extrude, revolve, fillet, chamfer, hole, loft, sweep, shell, draft, thread, linear/circular pattern, mirror, import STL)	7
M2.4	4-stage cascade constraint solver: DogLeg → LM → BFGS → NR with 19 constraint types	8
M2.5	STL/OBJ import via `wasm-meshkit`; binary + ASCII detection	8
M2.6	STEP import/export via `truck-stepio`	9
M2.7	Undo/redo history with delta snapshots	9
M2.8	GPU tessellation pipeline; adaptive LOD	10

Exit Criteria: Can model a bracket with holes, fillets, and chamfers; STEP round-trip preserves topology.

Phase 3: Manufacturing Intelligence (Weeks 11–16)

✓ Phase Complete

CAM engine with 8 post-processors (Fanuc, Haas, Mazak, Heidenhain, Grbl, LinuxCNC, Marlin, Klipper), DFM analyzer (27 tests), nesting engine, and cost estimation pipeline. 64 CAM tests passing.

Goal: Implement DFM analysis, cost estimation, toolpath generation, and sheet metal operations.

Milestone	Deliverables	Week
M3.1	DFM rule engine: thin walls, draft angles, undercuts, sharp edges	11
M3.2	Material catalog with 17 metals + 6 plastics; physical properties	11
M3.3	Cost estimation: Kienzle cutting force, Taylor tool life	12
M3.4	Sheet metal: bend allowance (K-factor), flat pattern unfolding	13
M3.5	Laser/plasma/waterjet cutting simulation with kerf compensation	13
M3.6	Nesting engine: bottom-left fill + parallel rotation search	14
M3.7	Toolpath generation: roughing, finishing, contouring	15
M3.8	G-code post-processors: Fanuc, Haas, Mazak, Heidenhain, Grbl, LinuxCNC, Marlin, Klipper	16

Exit Criteria: Upload STL → auto-DFM → cost quote → nested layout → G-code download workflow complete.

Phase 4: Desktop & Plugins (Weeks 17–22)

i Phase In Progress

Tauri desktop app functional with 37 IPC commands, React frontend with Zustand store (1,062 lines, 319 tests), Three.js viewport, and complete sketch tool system (7 tools, StatefulTool pattern). Plugin runtime operational (wasmtime 28, 7 tests). Remaining: view modes, auto-updater, packaging.

Goal: Polish the desktop application, implement the plugin system, and release the first public beta.

Milestone	Deliverables	Week
M4.1	egui viewport: 3D navigation (orbit, pan, zoom), selection, gizmos	17
M4.2	React panels: feature tree, properties, material picker, timeline	18
M4.3	8 view modes: Shaded, Wireframe, X-Ray, DFM Heatmap, etc.	19
M4.4	WASM plugin runtime (wasmtime 28); plugin manifest + security	20
M4.5	3 example plugins: material lib, DFM rule pack, post-processor	20
M4.6	Keyboard shortcuts, command palette, preferences system	21
M4.7	Auto-updater (Tauri); MSI/DMG/DEB packaging	21

© 2026 — MIT OR Apache-2.0

Milestone	Deliverables	Week
M4.8	Public beta release; documentation site; feedback channel	22

Exit Criteria: Desktop app installable on Windows/macOS/Linux; plugin SDK published; beta testers onboarded.

Phase 5: Cloud & Marketplace (Weeks 23–30)

Goal: Deploy the cloud backend, launch the plugin marketplace, and enable e-commerce integrations.

Milestone	Deliverables	Week
M5.1	Cloud API: JWT auth, file upload, job queue, SSE streaming	23
M5.2	Worker fleet: CAD, Analysis, CAM workers with Redis Streams	24
M5.3	Kubernetes Helm chart; horizontal auto-scaling	25
M5.4	Shopify + WooCommerce platform adapters	26
M5.5	Plugin marketplace: submit, review, install, update lifecycle	27
M5.6	Collaboration: project sharing, team workspaces, audit log	28
M5.7	Performance hardening: benchmark baselines, regression CI gate	29
M5.8	GA release; SLA commitments; on-call rotation established	30

Exit Criteria: Cloud API at <100ms p99; Shopify integration live; GA release published.

Risk Mitigation

Risk	Severity	Mitigation	Contingency
Truck B-Rep limitations	High	Early prototype of complex models; identify gaps by Week 6	Fall back to OpenCascade FFI for unsupported operations
wgpu driver bugs	Medium	Test on AMD/NVIDIA/Intel/Apple; use `wgpu` validation layer	Software rasterizer fallback for CI; reduce shader complexity
WASM plugin performance	Medium	Benchmark wasmtime AOT vs. interpreted; set memory limits	Native plugin API for performance-critical extensions
STEP parser coverage	Medium	Test against NIST STEP validation suite (100+ files)	OpenCascade STEP reader as fallback import path

© 2026 — MIT OR Apache-2.0

Risk	Severity	Mitigation	Contingency
Cloud scaling costs	Low	Auto-scaling with floor/ceiling limits; spot instances for workers	Async job queue with configurable concurrency caps

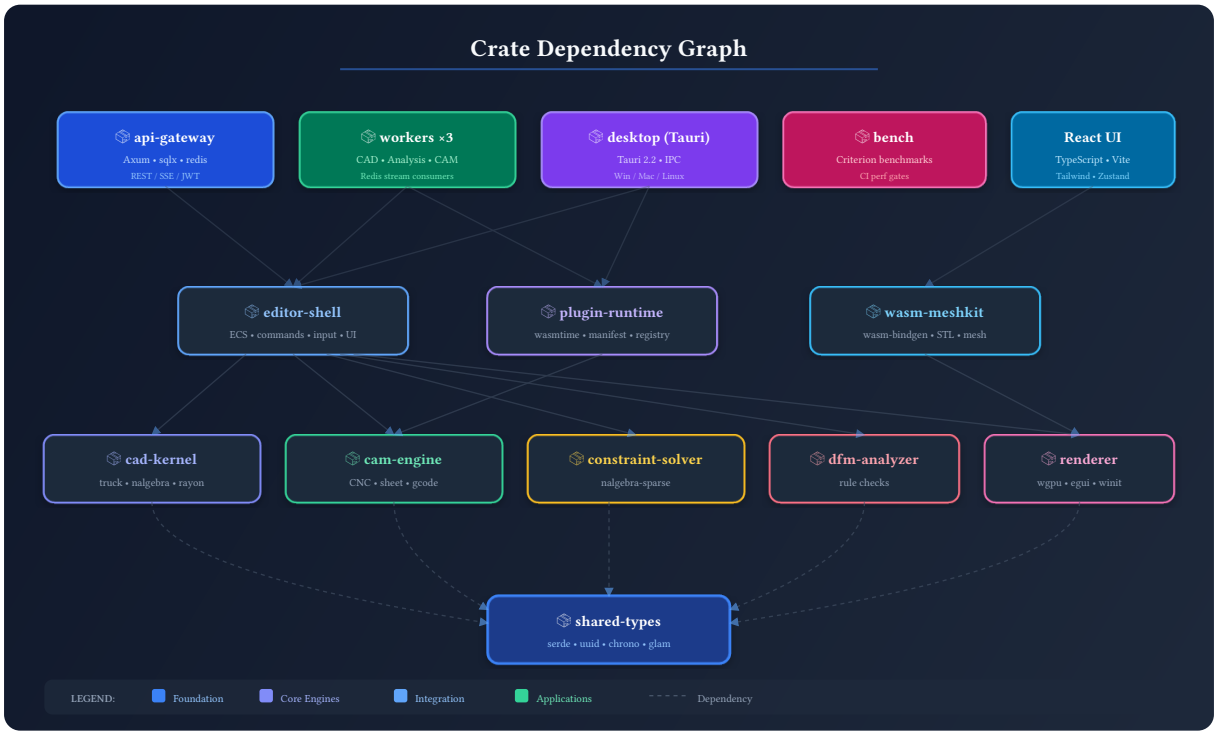
i Migration from Legacy Systems

Organizations migrating from existing DFM tools can use the **Bridge Mode**: import legacy material databases, DFM rule sets, and machine profiles via CSV/JSON bulk import endpoints. A migration wizard guides users through mapping legacy fields to r3ditor's data model.

© 2026 — MIT OR Apache-2.0

Appendices

Appendix A: Workspace Crate Reference



The complete Cargo workspace comprises 15 crates organized in four layers:

Crate	Path	Lines / Tests	Description
shared-types	packages/shared-types	1,130 / —	Cross-cutting data types: geometry, materials, DFM, manufacturing, estimation, units, API contracts
cad-kernel	packages/cad-kernel	10,269 / 125	Truck B-Rep kernel, 20 feature types, sketch system (7 entities, tools, snap), tessellation, history
cam-engine	packages/cam-engine	1,555 / 64	CNC physics, sheet metal, toolpath, nesting, 8 G-code post-processors
constraint-solver	packages/constraint-solver	1,915 / 30	4-stage cascade solver (Dog-Leg→LM→BFGS→NR), 19 constraint types, 3D assembly solver

Crate	Path	Lines / Tests	Description
dfm-analyzer	packages/dfm-analyzer	561 / 27	Design-for-manufacturability analysis, rule engine, severity scoring
renderer	packages/renderer	553 / —	Rendering infrastructure: scene management, camera, mesh display
editor-shell	packages/editor-shell	1,301 / 22	World struct, 22 EditorCommand variants, 21 tool types, StatefulTool lifecycle
plugin-runtime	packages/plugin-runtime	463 / 7	wasmtime 28 plugin host, manifest parsing, capability-based security
wasm-meshkit	packages/wasm-meshkit	531 / 20	WASM mesh toolkit: STL parsing, mesh simplification, geometry queries
api-gateway	packages/api-gateway	— / —	Axum 0.7 REST API, JWT auth, file upload, job queue, SSE
worker-analysis	packages/worker-analysis	— / —	Redis Stream consumer for DFM analysis jobs
worker-cad	packages/worker-cad	— / —	Redis Stream consumer for geometry import jobs
worker-cam	packages/worker-cam	— / —	Redis Stream consumer for toolpath/nesting computation
bench	packages/bench	— / —	Criterion benchmark suite: tessellation, constraints, CNC, nesting
desktop	apps/desktop/src-tauri	800 / —	Tauri 2.2 shell: main.rs + commands.rs (37 IPC handlers, 692 lines)

Appendix B: Technology Matrix

Technology Matrix

Category	Technology	Version	Purpose
Language	Rust 2021 + TypeScript 5.6	Edition 2021	Core + Frontend
GPU	wgpu + naga (WGSL)	28.0	Render + Compute
CAD Kernel	Truck + OpenCASCADE (FFI)	0.6 / 0.2	B-Rep + NURBS
UI Framework	egui + React + Tailwind	0.31 / 18 / 3.4	Viewport + Panels
Desktop Shell	Tauri 2.2 + winit	2.2 / 0.30	Cross-platform
Backend	Axum + sqlx + Redis	0.7 / 0.7 / 0.25	API + Workers
Infrastructure	Docker + K8s + GitHub Actions	Latest	Deploy + CI/CD

© 2026 — MIT OR Apache-2.0

Technology	Version	Purpose
Rust	1.93+	Primary systems language
TypeScript	5.6+	Frontend application language
React	18.3	UI component framework
Tailwind CSS	3.4+	Utility-first CSS framework
Vite	6.4.1	Frontend build tool
Tauri	2.2	Desktop shell (Rust + WebView)
Three.js	0.170	3D rendering in webview viewport
Zustand	5.0	React state management
truck	0.6	B-Rep geometry kernel (pure Rust)
nalgebra	0.33	Linear algebra and sparse matrices
glam	0.29	Fast 3D math (Vec3, Mat4)
Axum	0.7	Async HTTP framework
SQLx	0.8	Async PostgreSQL driver
Tokio	1.42	Async runtime
Rayon	1.10	Data parallelism (work-stealing)
wasmtime	28.0	WASM plugin runtime
serde	1.0	Serialization framework
Vitest	3.2	Frontend test framework
PostgreSQL	16	Primary database
Redis	7	Job queue + session cache
MinIO	latest	S3-compatible object storage
Docker	27+	Containerization

Appendix C: Database Schema

Five primary tables store application data:

```sql
-- Users & Authentication
CREATE TABLE users (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email       VARCHAR(255) UNIQUE NOT NULL,
    name        VARCHAR(255) NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
```

```
    role        VARCHAR(50) DEFAULT 'user',
    created_at  TIMESTAMPTZ DEFAULT now(),
    updated_at  TIMESTAMPTZ DEFAULT now()
);

-- Projects (CAD models)
CREATE TABLE projects (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id     UUID REFERENCES users(id) ON DELETE CASCADE,
    name        VARCHAR(255) NOT NULL,
    description TEXT,
    file_key    VARCHAR(512),  -- MinIO object key
    file_size   BIGINT,
    format      VARCHAR(50),   -- 'manu', 'step', 'stl', etc.
    thumbnail   VARCHAR(512),  -- MinIO thumbnail key
    created_at  TIMESTAMPTZ DEFAULT now(),
    updated_at  TIMESTAMPTZ DEFAULT now()
);

-- Analysis Jobs
CREATE TABLE jobs (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    project_id  UUID REFERENCES projects(id) ON DELETE CASCADE,
    job_type    VARCHAR(50) NOT NULL,  -- 'dfm', 'cost', 'nest', 'gcode'
    status      VARCHAR(50) DEFAULT 'pending',
    params      JSONB DEFAULT '{}',
    result      JSONB,
    error       TEXT,
    started_at  TIMESTAMPTZ,
    completed_at TIMESTAMPTZ,
    created_at  TIMESTAMPTZ DEFAULT now()
);

-- Material Catalog
CREATE TABLE materials (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name        VARCHAR(255) UNIQUE NOT NULL,
    category    VARCHAR(50) NOT NULL,  -- 'metal', 'plastic', 'composite'
    density     FLOAT NOT NULL,        -- g/cm³
    tensile_strength FLOAT,            -- MPa
    yield_strength   FLOAT,            -- MPa
    thermal_conductivity FLOAT,        -- W/m·K
    hardness    FLOAT,                 -- HB
    cost_per_kg FLOAT,                 -- USD
    specific_cutting_force FLOAT,      -- N/mm² (Kienzle kc1.1)
    taylor_n    FLOAT,                 -- Taylor exponent
    properties  JSONB DEFAULT '{}',    -- Extended properties
    created_at  TIMESTAMPTZ DEFAULT now()
);

-- Quotes
CREATE TABLE quotes (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    project_id  UUID REFERENCES projects(id) ON DELETE CASCADE,
    material_id UUID REFERENCES materials(id),
    quantity    INTEGER NOT NULL DEFAULT 1,
    unit_cost   FLOAT,
    setup_cost  FLOAT,
    material_cost FLOAT,
```

```
    total_cost  FLOAT,
    lead_time_days INTEGER,
    breakdown   JSONB DEFAULT '{}',    -- Detailed cost breakdown
    created_at  TIMESTAMPTZ DEFAULT now()
);

-- Indexes
CREATE INDEX idx_projects_user ON projects(user_id);
CREATE INDEX idx_jobs_project ON jobs(project_id);
CREATE INDEX idx_jobs_status ON jobs(status);
CREATE INDEX idx_materials_category ON materials(category);
```

Appendix D: Docker Services Inventory

Service	Image	Port	CPU	Memory
api-gateway	r3ditor/api:latest	3001	0.5	512 MB
worker-cad	r3ditor/worker-cad:latest	—	2.0	1 GB
worker-analysis	r3ditor/worker-analysis:latest	—	1.0	512 MB
worker-cam	r3ditor/worker-cam:latest	—	2.0	1 GB
postgres	postgres:16-alpine	5432	1.0	1 GB
redis	redis:7-alpine	6379	0.25	256 MB
minio	minio/minio:latest	9000/9001	0.5	512 MB
jaeger	jaegertracing/all-in-one	16686	0.25	256 MB
prometheus	prom/prometheus	9090	0.25	256 MB

Total resource requirements: 7.75 CPU cores, 5.3 GB RAM (development profile)

Appendix E: Platform Adapter Configuration

Shopify Adapter

```
[shopify]
api_version = "2024-10"
scopes = ["read_products", "write_products", "read_orders"]
webhook_topics = ["orders/create", "orders/paid"]
embed_mode = "app_bridge"  # iframe in Shopify admin
storefront_widget = true
```

WooCommerce Adapter

```
[woocommerce]
api_version = "wc/v3"
auth_method = "oauth1"     # Consumer key/secret
webhook_events = ["order.created", "order.completed"]
widget_shortcode = "[r3ditor_configurator]"
rest_namespace = "r3ditor/v1"
```

Appendix F: Keyboard Shortcuts Reference

Shortcut	macOS	Action
Ctrl+N	⌘+N	New project
Ctrl+O	⌘+O	Open project (.manu, .step, .stl)
Ctrl+S	⌘+S	Save project
Ctrl+Z	⌘+Z	Undo
Ctrl+Y	⌘+⇧+Z	Redo
Ctrl+Shift+E	⌘+⇧+E	Export (format dialog)
1–8	1–8	Switch view mode (Shaded, Wireframe, X-Ray, etc.)
F	F	Fit model to viewport
G	G	Toggle grid
Ctrl+Shift+A	⌘+⇧+A	Run DFM analysis
Ctrl+Shift+C	⌘+⇧+C	Generate cost estimate
Ctrl+Shift+P	⌘+⇧+P	Command palette
Delete	⌫	Delete selected feature
Escape	Escape	Cancel current operation / deselect
Middle Mouse	Middle Mouse	Orbit viewport
Shift+Middle Mouse	Shift+Middle Mouse	Pan viewport
Scroll Wheel	Scroll Wheel	Zoom viewport

© 2026 — MIT OR Apache-2.0

Appendix G: Glossary

Term	Definition
B-Rep	Boundary Representation — solid model defined by vertices, edges, faces, shells
DFM	Design for Manufacturability — analysis of part geometry for manufacturing feasibility
G-code	Numerical control programming language for CNC machine tools (ISO 6983)
IPC	Inter-Process Communication — Tauri's typed invoke system between React frontend and Rust backend
Kerf	Width of material removed by a cutting tool (laser, plasma, waterjet)
K-factor	Ratio of neutral axis position to sheet thickness for bend calculations
Nesting	Optimal arrangement of parts on a sheet to minimize material waste
PBR	Physically Based Rendering — lighting model using metallic/roughness workflow
PMI	Product Manufacturing Information — annotations embedded in 3D models (STEP)
SSE	Server-Sent Events — HTTP-based push protocol for real-time job status updates
StatefulTool	Blender-inspired tool lifecycle pattern with state machine (Idle ⟶ Active ⟶ Complete)
Sketch	2D drawing plane with entities (lines, circles, arcs) and geometric constraints
Snap	Automatic alignment to grid points, endpoints, midpoints, or intersections during drawing
Tessellation	Conversion of B-Rep surfaces to triangle meshes for rendering
WASM	WebAssembly — portable binary format for sandboxed plugin execution

© 2026 — MIT OR Apache-2.0